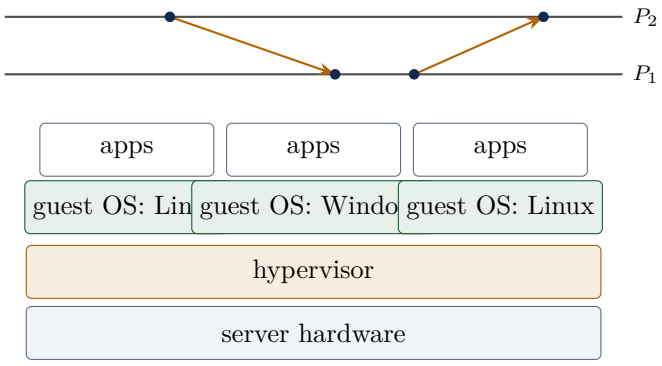
Cloud Computing

Virtualisation, Cloud Architecture and Time in Distributed Systems

An Insem 2 book for IT457: every idea explained, every numerical worked



Prepared by

Krushna Parmar

Roll No. **202611032** • Course IT457 • DA-IICT / DAU

Built on sir's Week 5–8 slides and the course textbook (D. Comer, *The Cloud Computing Book*), with Popek & Goldberg (1974), Lamport (1978), Tanenbaum & van Steen, Hwang–Fox–Dongarra and the Azure Architecture Center. Sir's slides appear as pictures; the text under each one explains it fully.

Contents

How to Read This Book **v**

I Virtualisation **1**

1 Elastic Computing and the Cloud Landscape **3**

1.1 The data center . . . 3
1.2 Why is it called “cloud” computing? . . . 4
1.3 Multi-tenancy . . . 4
1.4 Elastic computing . . . 5
1.5 Virtualised servers . . . 5
1.6 Service models: IaaS, PaaS, SaaS (and DaaS) . . . 6
1.7 Types of cloud: private, public, hybrid, multi-cloud . . . 8
1.8 Hyperscalers . . . 9

2 Virtual Machines, Hypervisors and the Types of Virtualisation **11**

2.1 What a virtual machine is . . . 11
2.2 Three approaches to virtualisation . . . 12
2.3 Types of virtualisation . . . 14
2.4 Hypervisors: type 1 and type 2 . . . 15
2.5 How a VM runs at hardware speed: privilege levels . . . 16
2.6 I/O: virtual devices . . . 17
2.7 A VM as a digital object, and migration . . . 17

3 Popek–Goldberg, Trap-and-Emulate, and the Requirements for Virtualisation **21**

3.1 The setting . . . 21
3.2 The three properties a VMM must have . . . 21
3.3 Classifying instructions . . . 22
3.4 Theorem 1 . . . 22
3.5 Trap and emulate . . . 23
3.6 Why x86 was a problem, and the three fixes . . . 24

3.7 Requirements for virtualisation . . . 25
3.8 Connecting everything . . . 26

4 The Virtual Private Cloud (VPC) 27
4.1 Definition . . . 27
4.2 The building blocks . . . 27
4.3 The standard picture . . . 29
4.4 Numericals on addresses . . . 29

II Cloud architecture 31

5 Cloud Architecture: Design Goals, Layers and Challenges 33
5.1 Design goals of a cloud platform . . . 33
5.2 A generic security-aware cloud architecture . . . 34
5.3 The layered cloud architecture . . . 34
5.4 Market-oriented cloud architecture . . . 35
5.5 Infrastructure requirements for virtualisation . . . 36
5.6 Six architectural design challenges . . . 37

6 Types of Cloud Architecture (Architecture Styles) 39
6.1 The five styles in one picture each . . . 39
6.2 Comparison . . . 40

7 N-tier Architecture 41
7.1 Definition: layers and tiers . . . 41
7.2 The three-tier architecture . . . 43
7.3 When to use N-tier, benefits, challenges . . . 44
7.4 Reference designs . . . 44
7.5 Multi-region deployment and Traffic Manager . . . 46
7.6 How to develop an N-tier architecture: sir's five steps . . . 47
7.7 Drawing an N-tier architecture on paper . . . 48
7.8 Worked design questions . . . 49
7.9 Numericals . . . 49

8 The Web-Queue-Worker Architecture 53
8.1 The problem it solves . . . 53
8.2 The architecture . . . 53
8.3 Why "purely PaaS" . . . 54
8.4 When to use, benefits, challenges . . . 54
8.5 Web-Queue-Worker vs N-tier . . . 55
8.6 Numericals . . . 56

III Distributed systems: time **59**

9 Time in Distributed Systems: Physical Clocks and Synchronisation **61**
9.1 Why clocks matter . . . 61
9.2 Skew, drift, precision, accuracy . . . 62
9.3 UTC . . . 64
9.4 Cristian's algorithm . . . 64
9.5 The Network Time Protocol (NTP) . . . 66
9.6 The Berkeley algorithm . . . 68
9.7 Comparing the three algorithms . . . 70

10 Logical Clocks: Lamport and Vector Clocks **71**
10.1 The happened-before relation . . . 71
10.2 Lamport's logical clocks . . . 72
10.3 Worked example: the running diagram . . . 74
10.4 Partial order, total order, and totally ordered multicast . . . 75
10.5 Vector clocks . . . 76

IV Exam kit **81**

11 Insem 2 Exam Kit **83**
11.1 Formula sheet . . . 83
11.2 Slips on the slides, all in one place . . . 83
11.3 Practice numericals (answers verified in `scripts/numericals.py`) . . . 84
11.4 Conceptual question bank . . . 85
11.5 Design questions to practise drawing . . . 87
11.6 Last-night summary . . . 87
11.7 Sources . . . 87

How to Read This Book

The syllabus for Insem 2 has three blocks:

Block	Topics
Virtualisation	VM, VPC, types of virtualisation, hypervisors, Popek–Goldberg theorem, trap and emulate, requirements for virtualisation
Cloud architecture	basics, types of cloud architecture, N-tier, Web-Queue-Worker
Distributed systems	timing and synchronisation, Lamport (logical) clocks, vector clocks

Questions are **conceptual** (from the lecture notes and reading material) and **numerical**. This book is built around both. Every chapter has the same rhythm: sir's slide as a picture, then the idea in plain words, then the precise statement, then a diagram, then worked numericals, then the questions that are likely to be asked.

The boxes

Numerical — orange panel.
A problem of the kind that appears in the exam, solved line by line. Every number is recomputed and asserted in `scripts/numericals.py`.

How to draw it in the exam — purple panel.
A step-by-step recipe for a diagram you will be asked to draw on paper (N-tier, VPC, Web-Queue-Worker, space–time diagrams).

Derivation — blue panel.
A formula built from scratch.

Mental model — green panel.
The picture to keep in your head.

Watch out — red panel.
A common mistake, or a slip on a slide.

Check yourself — purple rule.
Short questions with answers.

Where each slide is explained

Deck	Slides	Chapter
Week 5 Lec 1 (Cloud intro)	1–33	Ch. 1
Week 5 Lec 2 (Virtual machines)	1–15	Ch. 2, 3
Week 6 Lec 1 (Cloud architecture)	1–8, 18	Ch. 5
	9	Ch. 6
	10–17	Ch. 7
Week 8 Lec 2 (Synchronisation)	1–6, 19–25	Ch. 9
	7–18	Ch. 10
Friends' notes (VPC, P&G, trap–emulate; study notes)	all	Ch. 4, 3, 11

Part I

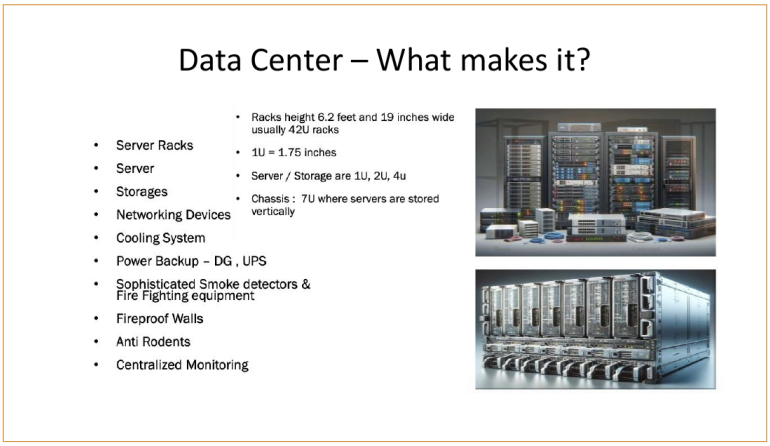
Virtualisation

1

Elastic Computing and the Cloud Landscape

Before virtual machines make sense, you need the reason they exist. This chapter follows sir's Week 5 Lecture 1 (textbook chapters 2 and 3): what a data center is, why many customers share one, what "elastic" means, how the three service models split responsibility, and the four ways an organisation can use the cloud.

1.1 The data center

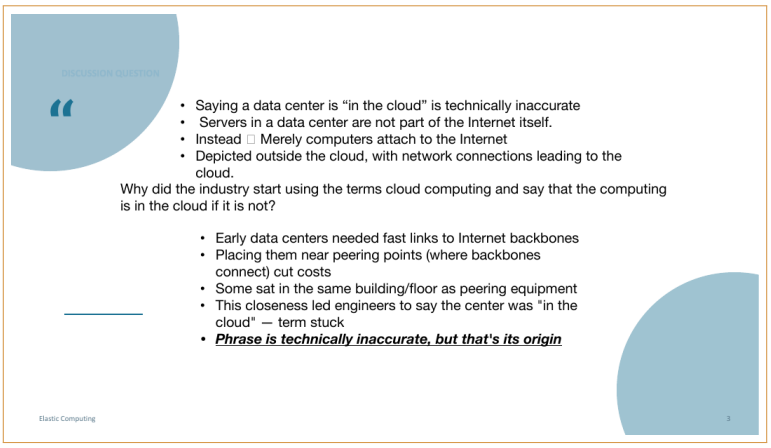


Sir's slide (w5a-04). What a data center is made of.

A **data center** is a dedicated physical facility that houses an organisation's critical applications, data and IT infrastructure. Physically it is rows of **racks** (a standard rack is about 6.2 ft tall and 19 in wide, holding 42 units; one unit, 1U, is 1.75 in), filled with **servers**, **storage**, and **network switches**, plus everything that keeps them alive: cooling, power backup (diesel generators, UPS), fire suppression and smoke detection, fireproof walls, and centralised monitoring.

A server in a data center is just a computer without a screen. What makes the facility special is scale and reliability: thousands of servers, redundant power, redundant cooling, fast links to the Internet.

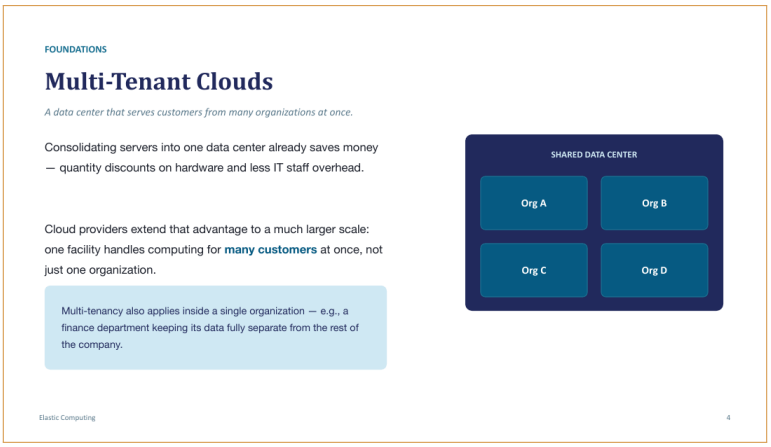
1.2 Why is it called “cloud” computing?



Sir’s slide (w5a-07). The origin of the word.

Network diagrams draw the Internet as a cloud. Saying a data center is “in the cloud” is technically inaccurate: servers are not part of the Internet, they are computers *attached* to it. The phrase stuck because early data centers were built next to **peering points** (where Internet backbones meet) to get fast, cheap links, sometimes on the same floor as the peering equipment. Engineers began saying the computing was “in the cloud”, and the name stayed.

1.3 Multi-tenancy



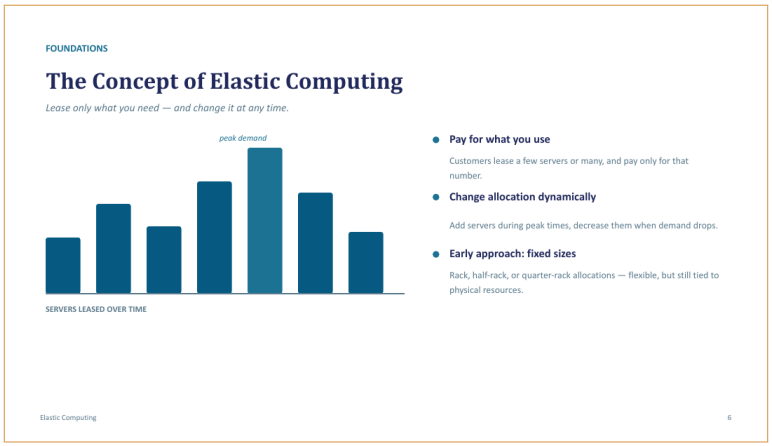
Sir’s slide (w5a-09). One facility, many organisations.

A **multi-tenant** data center serves customers (tenants) from many organisations at once. Consolidating servers into one facility already saves money (bulk hardware discounts, fewer staff); a cloud provider pushes the same idea to a much larger scale. Multi-tenancy also exists *inside* one organisation: the finance department’s data kept fully separate from everyone else’s.

Watch out

Multi-tenancy only works if tenants are **isolated**: no tenant may see or affect another’s data or computation. Isolation is exactly what virtualisation provides (Chapter 2) and what a VPC provides for networks (Chapter 4).

1.4 Elastic computing



Sir's slide (w5a-11). Lease only what you need, change it at any time.

> **Elastic computing**: a customer leases as many servers as it needs, pays only for those, and changes the number at any time: more during peaks, fewer when demand falls.

Early clouds leased fixed physical units (a rack, half rack, quarter rack): flexible, but still tied to hardware. Modern clouds lease *virtual* servers, which can be created in milliseconds.

Numerical

Question. A seasonal company needs 100 servers for 5 peak months and 20 servers for the other 7 months. (a) If it owns enough servers for the peak, what is its average utilisation? (b) Owning costs Rs. 300 per server-month; leasing costs Rs. 450 per server-month. Which is cheaper per year?

Solution. (a) Average servers actually needed $= \dfrac{5 \times 100 + 7 \times 20}{12} = \dfrac{640}{12} = 53.3$. Owned fleet $= 100$, so utilisation $= 53.3/100 = \mathbf{53.3}\%$: almost half the fleet idles all year.
(b) Owning: $100 \times 12 \times 300 = Rs.\,3{,}60{,}000$. Leasing: 640 server-months $\times 450 = Rs.\,2{,}88{,}000$. Leasing saves Rs. 72,000 even though each server-month costs 50% more, because the company stops paying for idle machines. That is the economic argument for elasticity in one line.

1.5 Virtualised servers



Sir's slide (w5a-13). Three properties of virtualised servers.

A provider does not hand out physical machines. It runs software (a hypervisor) that creates **virtualised servers**, with three properties:

1. **Rapid creation and removal**: purely software, no reboot of physical machines.
2. **Physical sharing**: many virtual servers run on one physical server at the same time.
3. **Logical isolation**: no tenant can observe or affect another's data or computation.



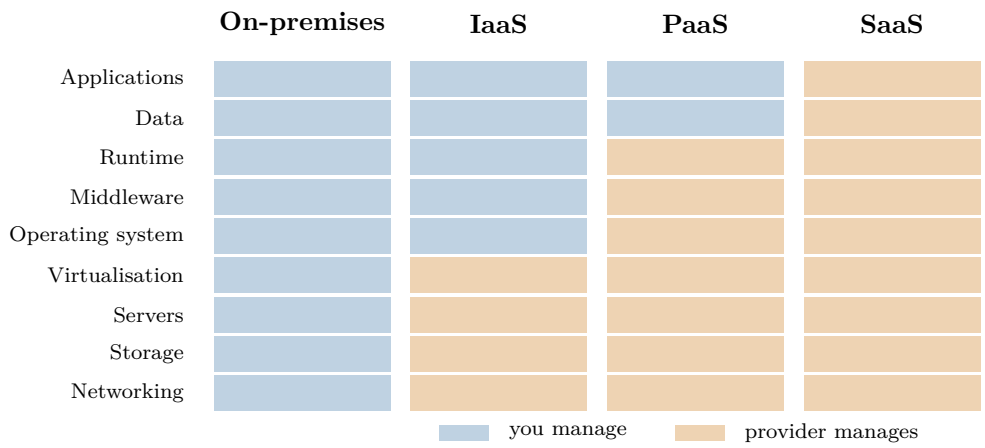
Sir's slide (w5a-15). Who benefits.

Provider: creation in milliseconds, no physical reconfiguration, tenants can be mixed on one server safely, and new VMs can be placed on lightly loaded machines (load balancing). **Customer**: a VM behaves like a real server with its own IP address, services are easy to deploy, capacity grows by adding copies of an app, and new software can be tested in isolation.

1.6 Service models: IaaS, PaaS, SaaS (and DaaS)



Sir's slide (w5a-17). As you move up the stack, the provider manages more.



Separation of responsibilities (the same picture as sir's slide 18).

- **IaaS** (Infrastructure as a Service): the provider runs the building, power, cooling, servers, network and block storage, and boots physical or virtual servers. The customer chooses the OS, applications and firewall rules. Often comes with load balancers, backup and network security; advanced offerings auto-scale. *Example: AWS EC2, Azure VMs, Google Compute Engine.*

- **PaaS** (Platform as a Service, formerly "Framework as a Service"): the provider also runs the OS, runtimes (Java, .NET), compilers and middleware; the customer only builds and deploys code. *Example: Azure App Service, Google App Engine, Heroku.*

- **SaaS** (Software as a Service): finished applications by subscription. The customer manages only the account. *Example: Office 365, Gmail, Salesforce CRM.*

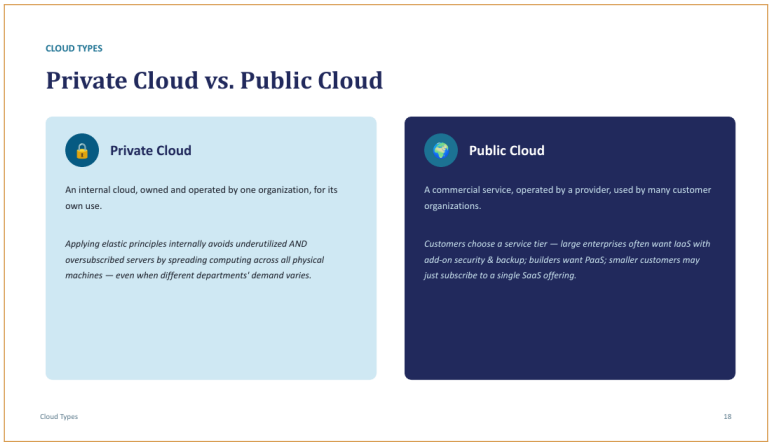
- **DaaS** (Desktop as a Service): a special SaaS that delivers a whole remote desktop (OS, apps and files in the cloud).



Sir's slide (w5a-22). SaaS guarantees.

SaaS promises three things: **universal access** (any device, app or browser), **guaranteed sync** (one copy of each file, so a change on the laptop appears on the phone) and **high availability** (backup power, off-site backups).

1.7 Types of cloud: private, public, hybrid, multi-cloud



Sir's slide (w5a-24). Private vs public.

	Private cloud	**Public cloud**
Owner	one organisation, for itself	provider, for many customers
Why	control and visibility (root-cause access, regulation), low latency on-premises, insurance against rate hikes	economy of scale (up to ∼75% cheaper), expertise (incl. AI/ML), advanced services (AIops, platforms)
Risk	capital cost, under-used hardware	provider **lock-in**

Providers save money with bulk discounts, shared staff, and cheap **whitebox switches** controlled by **SDN** (software-defined networking): a controller computer installs forwarding tables in the switches. **Lock-in** means the enticements (free convenience services, migration tools) and obstacles that make leaving a provider expensive.



Sir's slide (w5a-30). Hybrid and multi-cloud.

- **Hybrid cloud**: some computing on a private cloud, the rest on a public one. Reasons: *control when needed* (classified or regulated data stays private) and *computation overflow* (send peak-season load to the public cloud instead of buying idle servers). Cost: two security systems to learn and run.
- **Multi-cloud**: more than one *public* provider, to avoid lock-in (e.g. business units on different providers). Cost: services differ, moving data is hard, custom software may be needed to combine outputs.

1.8 Hyperscalers



Sir's slide (w5a-32). The largest providers.

Hyperscalers own the largest cloud facilities. Today the definition is about infrastructure spend: by 2018 Google, Amazon, Microsoft, Facebook and Alibaba together spent about $11B a year on new servers. The three enterprise leaders are **AWS** (launched 2006; $25.7B, 11% of Amazon's revenue in 2018), **Azure** (2010; Microsoft cloud revenue $32.2B, 29% in 2018) and **GCP**.

Watch out

Sir's slide 32 places the $32.2B under Azure and $25.7B under GCP. In the textbook (Comer, §3.10) $32.2B is Microsoft's cloud revenue and $25.7B is *AWS* revenue, both for 2018. Quote it the textbook's way.

Check yourself

Q. Name the three properties of virtualised servers.
A. Rapid creation/removal, physical sharing, logical isolation.

Q. IaaS or PaaS: you choose the operating system.
A. IaaS.

Q. Two reasons for hybrid cloud?
A. Control when needed (regulated data private) and computation overflow.

Q. What is lock-in?
A. Enticements and obstacles that make moving to another provider inconvenient or costly.

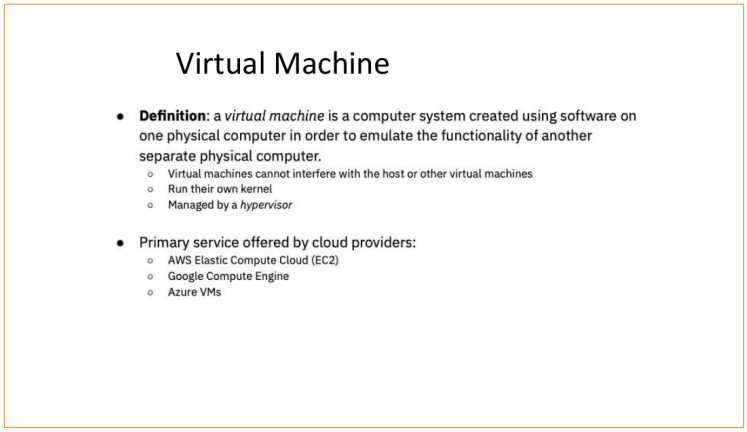
Q. NIST's five essential characteristics of cloud?
A. On-demand self-service, broad network access, resource pooling, rapid elasticity, measured service (NIST SP 800-145).

2

Virtual Machines, Hypervisors and the Types of Virtualisation

This chapter is sir's Week 5 Lecture 2 and textbook Chapter 5 (Comer, *Virtual Machines*), filled out with the standard classifications every exam on virtualisation expects.

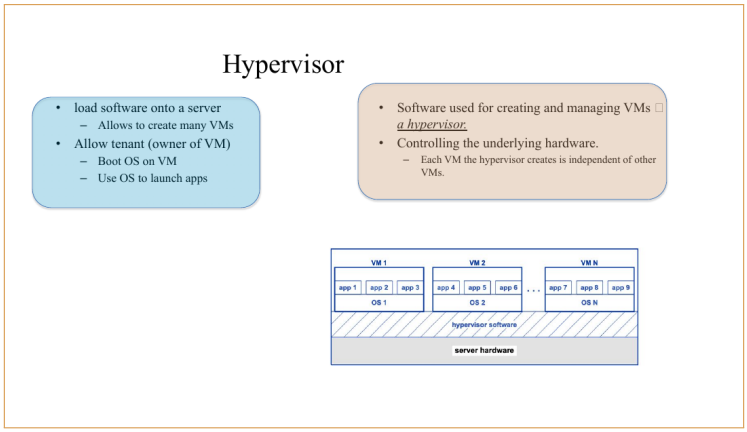
2.1 What a virtual machine is



Sir's slide (w5b-03). Definition of a virtual machine.

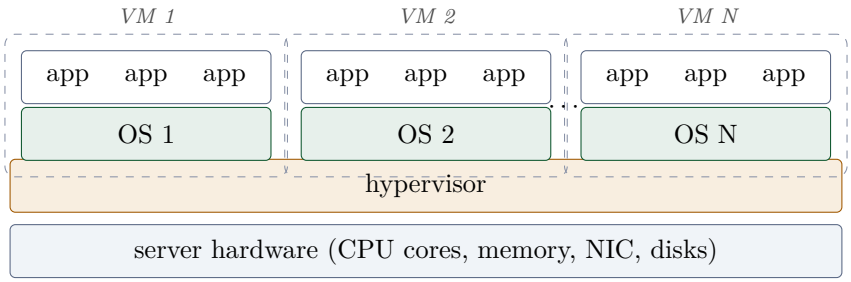
> A **virtual machine (VM)** is a computer system created by software on one physical computer so that it behaves like a separate physical computer. A VM cannot interfere with the host or with other VMs, runs its own kernel (its own operating system), and is managed by a **hypervisor**.

Cloud examples: AWS EC2, Google Compute Engine, Azure Virtual Machines.



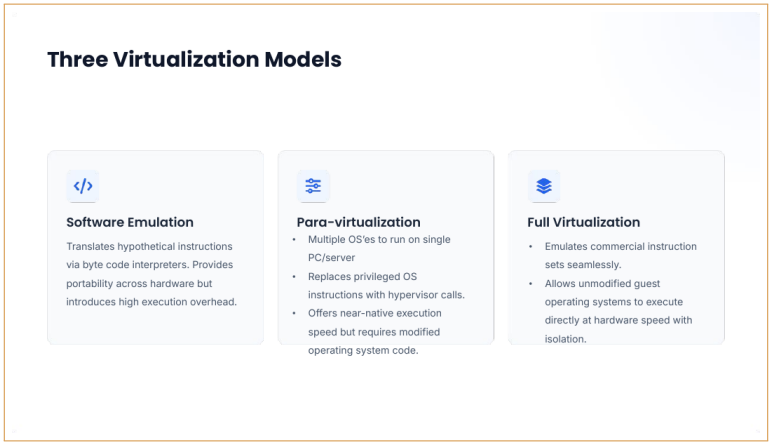
Sir's slide (w5b-04). The hypervisor.

The **hypervisor** (also called a **virtual machine monitor, VMM**) is the software that creates and manages VMs and controls the underlying hardware. Each VM it creates is independent of the others. The tenant (owner of the VM) boots an OS on it and uses that OS to launch apps.



Comer Fig. 5.1: a server running a (type 1) hypervisor and N VMs.

2.2 Three approaches to virtualisation



Sir's slide (w5b-06). Software emulation, para-virtualisation, full virtualisation.

1. Software emulation

A program (the **emulator**) treats another program's instructions as *data*: it reads one instruction, does what the target machine would do, moves to the next. This lets a program compiled for computer C_2 run on computer C_1. Language designers use it: compile to **byte code** for

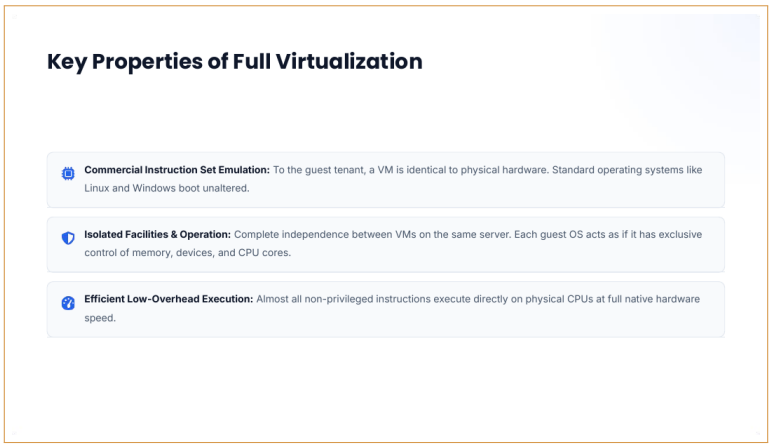
a hypothetical machine, then write a **byte-code interpreter** for each real machine (the Java Virtual Machine is the famous case). *Advantage*: portability. *Drawback*: every guest instruction costs many host instructions, so it is far too slow to be the main form of virtualisation in a data center.

2. Para-virtualisation

Pioneered in the 1960s. A hypervisor runs several operating systems at once and moves the processor among them, the way an OS moves the CPU among processes. Instructions run **natively** (directly on the CPU), so it is fast. But the guest OS source code must be **modified**: every privileged instruction (I/O, control of the hardware) is replaced by a call to the hypervisor, a **hypercall**. The hypervisor then performs the operation or refuses. *Example*: Xen with modified Linux (Barham et al., 2003).

3. Full virtualisation

Several OSes at once, at high speed, **without modifying** the OS. This is what clouds use today.



Sir's slide (w5b-09). Key properties of full virtualisation.

Three properties (Comer §5.3):

1. **Emulation of commercial instruction sets**: to the tenant a VM is identical to a real computer; unmodified Windows or Linux boots on it.

2. **Isolated facilities and operation**: each guest OS believes it owns all of memory, the I/O devices and every CPU core, yet VMs are completely isolated.

3. **Efficient, low-overhead execution**: almost all instructions, especially application code, run at full hardware speed.

These are the Popek–Goldberg properties in different words (Chapter 3).

	Emulation	Para-virtualisation	Full virtualisation
Guest OS modified?	no	**yes** (hypercalls)	no
Speed	slow (interpreted)	near native	near native
Guest ISA = host ISA?	need not be	must be	must be
How sensitive ops are caught	every instruction is software	guest calls hypervisor	trap and emulate, binary translation or hardware support
Examples	JVM, QEMU (TCG)	Xen PV	VMware ESXi, KVM, Hyper-V

Numerical

Question. A program executes 10^9 instructions, each taking 1 cycle natively. Of these, 0.02% are sensitive. Compare the run time under (a) software emulation at 25 host instructions per guest instruction, (b) trap-and-emulate costing 1000 cycles per sensitive instruction, (c) para-virtual hypercalls costing 300 cycles each.
Solution. Native time = 10^9 cycles. Sensitive instructions = $10^9 \times 0.0002 = 2 \times 10^5$.

(a) Emulation: 25×10^9 cycles, a **25**× slowdown.

(b) Trap-and-emulate: $10^9 + 2 \times 10^5 \times 1000 = 1.2 \times 10^9$, a **1.20**× slowdown (20% overhead).

(c) Para-virtual: $10^9 + 2 \times 10^5 \times 300 = 1.06 \times 10^9$, **1.06**×.

The lesson: what matters is *how often* the hypervisor must step in. Emulation steps in on every instruction; the other two only on the rare sensitive ones.

2.3 Types of virtualisation

"Types" is asked in two senses. Learn both.

(a) By technique (how the CPU is virtualised)

1. **Full virtualisation by binary translation**: the hypervisor scans guest kernel code and rewrites the troublesome sensitive instructions on the fly (VMware's original x86 method).

2. **Para-virtualisation**: the guest is modified to use hypercalls.

3. **Hardware-assisted virtualisation**: the CPU itself has a mode for the hypervisor (Intel VT-x, AMD-V, 2005–06), so sensitive instructions trap automatically (Chapter 3).

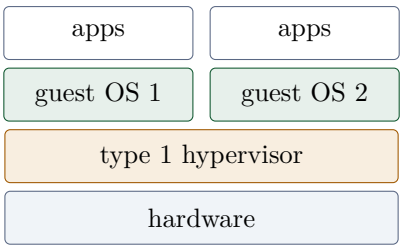
4. **OS-level virtualisation (containers)**: one kernel, many isolated user spaces (Docker, LXC). Not a VM: no separate kernel, so it is lighter but all containers share the host OS.

5. **Emulation**: a different ISA interpreted in software.

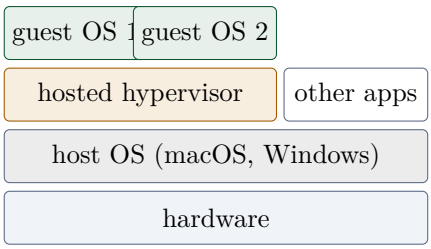
(b) By resource (what is virtualised)

Type	What it means	Example
Server / compute	many VMs on one physical server	EC2, ESXi, KVM
Memory	each VM sees its own "physical" memory; the hypervisor maps it (shadow page tables, EPT/NPT)	every hypervisor
Storage	disks are files or network volumes presented as local disks	EBS, virtual disk (Comer Fig. 5.4), SAN
Network	virtual NICs, switches and private networks on shared hardware	VPC, SDN, VLAN
Desktop	a user's whole desktop runs in the data center	DaaS, VDI, Windows 365
Application	an app runs in a sandbox without being installed on the OS	JVM, App-V
I/O	virtual devices, emulated or para-virtual, or direct pass-through	virtio, SR-IOV

2.4 Hypervisors: type 1 and type 2



Type 1: bare metal (data centers)



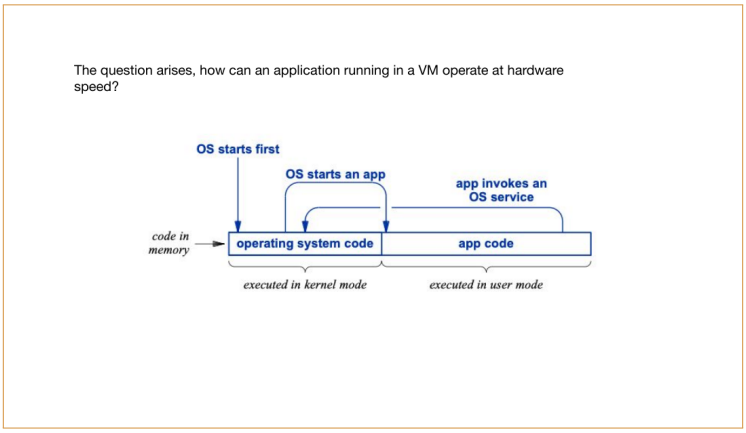
Type 2: hosted (laptops)

- **Type 1 (bare-metal / native)** runs directly on the hardware and owns it. Used in data centers. Examples: VMware ESXi, Xen, Microsoft Hyper-V, KVM (Linux kernel turned into a hypervisor).
- **Type 2 (hosted)** runs as an ordinary application on a host OS, in user space. Examples: VirtualBox (which you used in Lab 4), VMware Workstation/Fusion, Parallels. It needs two facilities (Comer §5.16): *processor support for virtualisation* (so a guest OS can behave as if it had full privilege without having it) and a *mapping of guest I/O onto host OS services* (guest network and disk requests become host OS calls; files can even be shared between guest and host).

Why a user wants type 2 (Comer §5.17): several OSes run *simultaneously* and you switch between them like windows, unlike multi-boot where only one OS runs at a time.

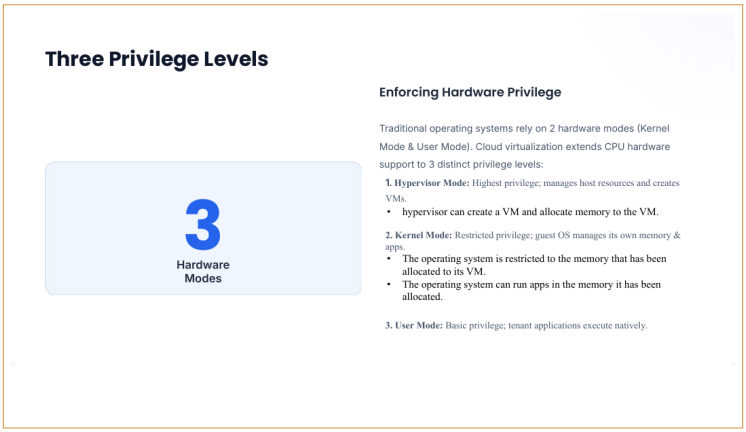
	Type 1	Type 2
Runs on	hardware	a host OS
Performance	higher (no host OS in the path)	lower (host OS adds a layer)
Isolation / security	stronger, small trusted base	depends on host OS
Use	cloud data centers, enterprise	development, testing, desktops

2.5 How a VM runs at hardware speed: privilege levels



Sir's slide (w5b-10). The question: how can an app in a VM run at hardware speed?

On a normal computer the CPU has two **modes**: **kernel mode** (OS, every instruction allowed) and **user mode** (apps, only basic instructions and their own memory). The CPU executes app code *directly*; the OS is not "in the way". If an app attempts a privileged instruction or touches memory it does not own, the processor raises an **exception** (a trap) and jumps to a fixed place in the OS. A system call is a deliberate trap.



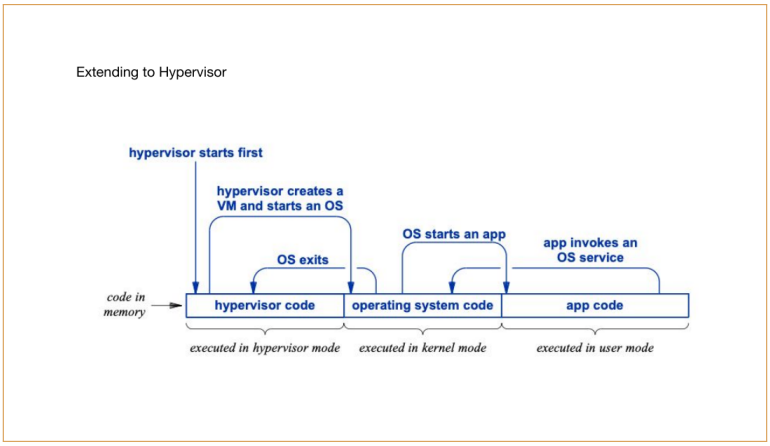
Sir's slide (w5b-12). Three privilege levels.

With virtualisation the CPU provides **three** levels:

1. **Hypervisor mode** (most privileged): only the hypervisor creates VMs and allocates memory to them.

2. **Kernel mode**: the guest OS runs here, restricted to the memory of its own VM.

3. **User mode**: tenant applications.



Sir's slide (w5b-13). Transitions between the three kinds of code in memory.

All three kinds of code sit in memory and run *directly* on the processor; only the mode changes. The sequence (sir's slide 14): (1) the hypervisor boots first in hypervisor mode and configures the hardware; (2) it creates a VM and starts the guest OS in kernel mode inside the VM's memory; (3) the guest launches an app in user mode, which runs at full speed; (4) a system call traps back to the guest kernel, and anything beyond the guest's privilege traps to the hypervisor.

Hierarchy of trust

Hypervisor > guest OS > app. Each level is trusted to configure the hardware correctly for the level below it (the hypervisor isolates VMs, the OS isolates apps). Any attempt to exceed your trust level passes control *up one level*. The hypervisor is trusted completely, so it must be small and correct.

2.6 I/O: virtual devices

A standard OS boots by scanning the bus and taking ownership of every device. A hypervisor cannot allow that: every VM needs the network card. Solution: **virtual I/O devices**. When the guest touches the bus, the access violates privilege, the hypervisor is invoked, runs the virtual device's software and replies *as if* the device had answered over the bus. Two designs (Comer §5.10):

- **Invent an imaginary device** with a clean interface (e.g. `virtio`): simpler, faster drivers, but each OS needs a new driver.
- **Emulate an existing device** (e.g. an Intel e1000 NIC): existing drivers work unchanged, but the software must mimic the real device exactly.

Example: a **virtual disk** looks like a local disk to the OS but sends every read and write over the data center network to the storage facility.

2.7 A VM as a digital object, and migration

Because the hypervisor records everything about a VM (memory regions, virtual devices, disk, status), a suspended VM can be packed into a set of bytes: a **digital object**. It can be saved, copied and **migrated** to another server. Uses: replace failing hardware, **balance load** (move

CPU-heavy VMs off a saturated CPU and I/O-heavy VMs off a saturated bus), power down idle servers at night, move a customer's VMs into one pod to cut latency.

Live migration in three phases

1. **Pre-copy**: copy all memory while the VM keeps running; record pages that change (dirty pages). Repeat on the dirty pages.
2. **Stop-and-copy**: pause the VM briefly and copy the pages still dirty.
3. **Post-copy**: send the remaining state (registers, device state) and resume on the new server.

Numerical

Question. A VM has 4096 MB of memory. The migration link carries 125 MB/s (1 Gb/s). The VM dirties memory at 25 MB/s. Pre-copy rounds continue until fewer than 64 MB remain dirty. Find the number of pre-copy rounds, the downtime, and the total migration time. Compare with plain stop-and-copy.
Solution. Each round copies what the previous round left dirty; while copying for t seconds the VM dirties $25t$ MB.

round	data to copy (MB)	time = data/125 (s)	dirtied meanwhile = $25t$ (MB)
1	4096	32.768	819.2
2	819.2	6.554	163.8
3	163.8	1.311	32.8

After round 3 only 32.8 MB $<$ 64 MB remain, so stop: **3 pre-copy rounds**. Stop-and-copy moves 32.8 MB in $32.8/125 = \mathbf{0.262}$ s: that is the **downtime**. Total $\approx 32.77+6.55+1.31+0.26 = \mathbf{40.9}$ s. Plain stop-and-copy would pause the VM for $4096/125 = \mathbf{32.8}$ s. Pre-copy trades a longer total time for a downtime 125 times shorter. Note each round shrinks by the ratio $D/B = 25/125 = 0.2$; if the dirty rate approached the bandwidth, pre-copy would never converge.

Numerical

Question. A host has 64 cores and 256 GB RAM; the hypervisor keeps 8 GB. Each VM needs 4 vCPUs and 16 GB. The provider over-commits CPU 2:1 but never memory. How many VMs fit?
Solution. By CPU: $64 \times 2/4 = 32$. By memory: $(256-8)/16 = 15.5 \Rightarrow 15$. The smaller wins: **15 VMs**, memory-bound. (CPU can be over-committed because VMs rarely use all cores at once; memory cannot be safely, because a page in use must exist somewhere.)

Check yourself

Q. Difference between para-virtualisation and full virtualisation?
A. Para needs a modified guest that makes hypercalls; full runs an unmodified guest.

Q. Why is software emulation unsuitable as the main form in a data center?
A. Every guest instruction costs many host instructions: large overhead.

Q. Name the three phases of live migration.

A. Pre-copy, stop-and-copy, post-copy.

Q. Is VirtualBox type 1 or type 2?

A. Type 2 (hosted).

Q. Is a container a VM?

A. No: containers share the host kernel; a VM runs its own kernel.

3

Popek–Goldberg, Trap-and-Emulate, and the Requirements for Virtualisation

Chapter 2 said a VM runs "almost every instruction at hardware speed" and that the hypervisor "steps in" for the rest. This chapter makes that precise. In 1974 Gerald Popek and Robert Goldberg asked: *for which processors can such a hypervisor be built at all?* Their answer is one line long and is the most likely theory question in this block.

3.1 The setting

Popek and Goldberg (*Formal Requirements for Virtualizable Third Generation Architectures*, CACM 17(7), 1974) model a "third generation" computer:

- the processor runs in **supervisor (system) mode** or **user mode**;
- memory is linear and accessed through a **relocation register** (base and bound): a program sees addresses $0..b$ and the hardware adds the base;
- some instructions are allowed only in supervisor mode;
- the machine state is $S = \langle E, M, P, R \rangle$: executable storage E, mode M, program counter P, relocation register R. (I/O and interrupts are left out of the model.)

A **VMM** is a program that runs in supervisor mode, puts each guest (including the guest's *own* OS) in **user mode**, and gives each guest its own region of memory through R.

3.2 The three properties a VMM must have

1. **Equivalence (fidelity)**: a program running under the VMM behaves essentially identically to how it behaves on the real machine (allowed differences: timing and available resources).
2. **Resource control (safety)**: the VMM is in complete control of the virtualised resources; a guest cannot use a resource the VMM did not give it, and the VMM can take resources back.

3. **Efficiency (performance)**: a statistically dominant fraction of the guest's instructions execute directly on the hardware, with no VMM intervention.

Equivalence: the guest cannot tell. Resource control: the guest cannot escape. Efficiency: it is not an emulator. Drop efficiency and an emulator satisfies the other two; drop resource control and you have no isolation. The theorem is about getting *all three* at once.

These are the same three properties as Comer's "properties of full virtualisation" (Chapter 2): emulation of the commercial instruction set = equivalence; isolated facilities = resource control; efficient low-overhead execution = efficiency.

3.3 Classifying instructions

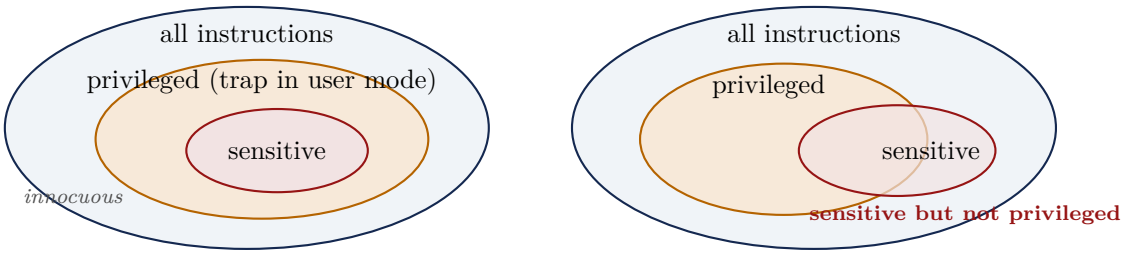
- **Privileged instruction**: *traps* if executed in user mode, and does not trap in supervisor mode.
- **Control-sensitive instruction**: tries to *change* the configuration of the system's resources: the mode, the relocation register, memory protection, I/O, the interrupt state.
- **Behaviour-sensitive instruction**: its *result depends* on that configuration: *location-sensitive* (depends on where in real memory the program is, i.e. on R) or *mode-sensitive* (depends on whether the CPU is in user or supervisor mode).
- **Sensitive** = control-sensitive **or** behaviour-sensitive. **Innocuous** = not sensitive.

Examples: `ADD`, `LOAD`, `JMP` are innocuous. "Load relocation register" and "set mode" are control-sensitive. "Read processor status (which includes the mode)" is mode-sensitive. "Store the real address of the page table" is location-sensitive.

3.4 Theorem 1

Theorem 1 (Popek and Goldberg). For any conventional third-generation computer, a VMM with equivalence, resource control and efficiency may be constructed if the set of **sensitive** instructions is a **subset** of the set of **privileged** instructions:

$$\text{Sensitive} \subseteq \text{Privileged}.$$



virtualisable by trap-and-emulate **not virtualisable by trap-and-emulate (old x86)**

Why the condition works (proof idea in four steps)

1. Run the guest (its apps *and* its OS) in real user mode, with R set to the guest's region.
2. Every **innocuous** instruction executes directly on the CPU: it cannot see or change anything outside the guest's region, so it behaves exactly as on the real machine. This gives **efficiency**, because innocuous instructions are the vast majority.
3. Every **sensitive** instruction is, by the hypothesis, privileged, so in user mode it **traps** to the VMM. The VMM looks at what the guest tried to do and applies it to the guest's *virtual* state (virtual mode, virtual relocation register): it **emulates** it. This gives **equivalence**.
4. Because nothing sensitive can execute without the VMM seeing it, the guest can never change the real mode, the real R, or touch another guest's memory. This gives **resource control**.

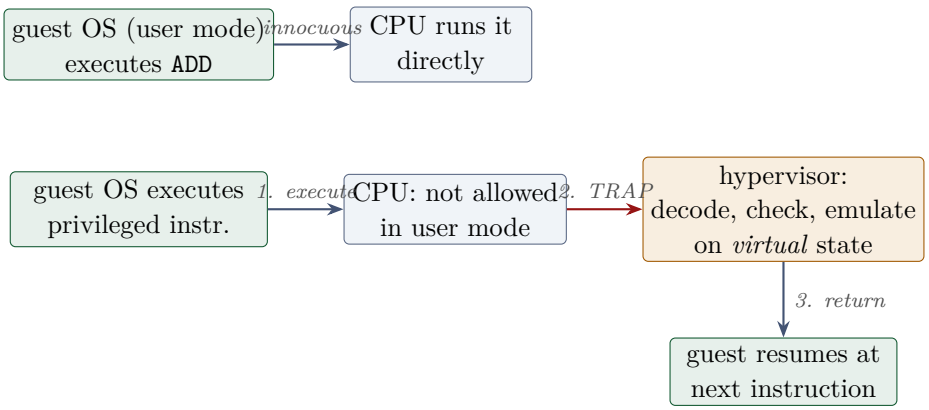
If even one sensitive instruction were *not* privileged, step 3 fails: the guest executes it silently, the VMM never knows, and either equivalence (the guest sees the wrong value) or resource control (the guest changes real state) breaks.

Theorems 2 and 3 (for completeness)

Theorem 2 (recursive virtualisation): a machine is recursively virtualisable (a VMM can run a copy of itself inside a VM) if it is virtualisable and a VMM without timing dependencies can be built. **Theorem 3 (hybrid VMM)**: a weaker *hybrid* VMM (which interprets all guest-supervisor-mode code in software and runs only guest-user code directly) can be built if the *user-sensitive* instructions are a subset of the privileged instructions.

3.5 Trap and emulate

> **Trap-and-emulate** is the classic virtualisation technique: the guest runs directly on the CPU in a less-privileged mode; when it executes a privileged (sensitive) instruction the CPU **traps** to the hypervisor, which **emulates** the effect on the guest's virtual state, then resumes the guest at the next instruction.



1. The guest kernel (thinking it is in kernel mode) executes, say, "disable interrupts".
2. The CPU is really in user mode, so the privileged instruction **traps**; control jumps to the hypervisor.
3. The hypervisor decodes the instruction, checks whether this VM may do it, and applies it to the VM's *virtual* CPU: it records "interrupts disabled" for VM 1 (it does *not* disable real interrupts, which other VMs need).
4. It advances the guest's program counter and resumes the guest, which believes the instruction ran.

A hotel guest wants something done in the electricity control room. They cannot walk in. Reception (the trap) stops them, checks the request, and the staff (the hypervisor) do the safe equivalent. The guest gets the effect they expected without ever touching the switchboard. Guest = guest OS, control room = hardware, reception = trap, staff action = emulation.

Why not trap everything? A billion traps per second would make the VM an emulator. Efficiency comes precisely from letting innocuous instructions run directly.

3.6 Why x86 was a problem, and the three fixes

The classic x86 (IA-32) violates Theorem 1: it has **sensitive instructions that do not trap in user mode**. Robin and Irvine (2000) listed 17; the usual list includes:

- **register reads**: `SGDT`, `SIDT`, `SLDT`, `SMSW`, `STR` read where the real descriptor tables are, or the machine status word, without trapping, so a guest learns it is not where it thinks it is (location-sensitive);
- `PUSHF`/`POPF`: `POPF` in user mode **silently ignores** a change to the interrupt-enable flag instead of trapping. A guest kernel that does `POPF` to re-enable interrupts gets no error and no effect, and the VMM is never told (mode-sensitive);
- protection checks: `LAR`, `LSL`, `VERR`, `VERW`, `PUSH`/`POP` of segment registers, `MOV` to/from segment registers, far `CALL`/`JMP`/`RET`, `INT n`: their result depends on the current privilege level.

Fix	Idea	Cost
Binary translation (VMware, 1999)	scan guest kernel code before it runs, rewrite each problem instruction into a safe sequence that calls the VMM; user code still runs directly	translation work, cache of translated code
Para-virtualisation (Xen, 2003)	change the guest OS source to call the hypervisor (hypercalls) instead of using problem instructions	guest must be modified
Hardware assist (Intel VT-x, AMD-V, 2005–06)	a new CPU mode: the hypervisor runs in *VMX root*, the guest kernel in *VMX non-root* ring 0; sensitive instructions cause a **VM exit** to the hypervisor, which returns with **VM entry**	each exit costs hundreds to a few thousand cycles

With VT-x/AMD-V, x86 meets the Popek–Goldberg requirements. Architectures that met them from the start: IBM System/370, PowerPC; RISC-V was designed for it. Ones that failed: PDP-10, Motorola 68000 (one instruction, `MOVE from SR`, fixed in the 68010).

3.7 Requirements for virtualisation

Put together, a platform can host efficient, safe VMs when:

1. **At least two CPU modes** (better three: hypervisor, kernel, user), so the hypervisor is more privileged than every guest.
2. **Sensitive ⊆ privileged** (Theorem 1), or a workaround: binary translation, para-virtualisation or hardware assist.
3. **Memory protection/relocation**: each guest confined to its own memory (relocation register in 1974; today an MMU with a second level of address translation, Intel EPT or AMD NPT, or shadow page tables).
4. **A trap mechanism** that transfers control to the hypervisor and back precisely (restartable instructions).
5. **I/O and interrupt virtualisation**: virtual devices, so no guest owns real devices (Chapter 2); interrupts routed by the hypervisor.
6. **The three properties**: equivalence, resource control, efficiency.

Numerical

Question. A hypothetical CPU has six instructions with these properties. Which are sensitive? Is the CPU virtualisable by trap-and-emulate? If not, name the culprit and a fix.

	instruction	traps in user mode?	changes config?	result depends on mode/location?
I1	`ADD`	no	no	no
I2	`SETMODE`	yes	yes	no
I3	`LDRR` (load reloc. reg.)	yes	yes	no
I4	`RDSTAT` (read status)	no	no	yes
I5	`HALT`	yes	yes	no
I6	`LOAD`	no	no	no

Solution. Sensitive = changes config *or* depends on it: {I2, I3, I4, I5}. Privileged = traps in user mode: {I2, I3, I5}. Is Sensitive ⊆ Privileged? No: **I4 is sensitive but not privileged** (behaviour-sensitive, mode-sensitive). A guest kernel running in user mode would read "user" while believing it is in kernel mode, and the VMM would never know: equivalence fails. So the CPU is **not** classically virtualisable. Fixes: make `RDSTAT` privileged in the next revision (as Motorola did in the 68010), use binary translation to replace `RDSTAT` in guest kernel code, or para-virtualise the guest so it asks the hypervisor for the mode.

Numerical

Question. A guest kernel executes 5×10^6 privileged instructions per second on a 3 GHz CPU. Each VM exit plus emulation costs 1500 cycles. What fraction of CPU time goes to the hypervisor? If para-virtualisation batches them into hypercalls of 50 operations each, costing 2000 cycles per hypercall, what is the fraction now?
Solution. Trap-and-emulate: $5 \times 10^6 \times 1500 = 7.5 \times 10^9$ cycles per second, more than the 3×10^9 available: the guest would spend all its time in exits and still fall behind. That is why frequent sensitive operations (e.g. page-table updates) are the worst case for pure trap-and-emulate. Batched hypercalls: $5 \times 10^6/50 = 10^5$ hypercalls $\times 2000 = 2 \times 10^8$ cycles, i.e. $2 \times 10^8/3 \times 10^9 = \mathbf{6.7}\%$. Batching is the main performance trick of para-virtualisation.

3.8 Connecting everything

Mental model

Popek–Goldberg is the *theory* (the condition sensitive $\subseteq$ privileged). **Trap-and-emulate** is the *technique* the condition makes possible. **Hardware assist, binary translation and para-virtualisation** are what you do when the condition fails. The **three properties** are what you get when it works.

Check yourself

Q. State the Popek–Goldberg theorem.
A. An efficient, safe, equivalent VMM can be built if every sensitive instruction is privileged.

Q. Define control-sensitive and behaviour-sensitive.
A. Control: changes the resource configuration. Behaviour: result depends on it (location or mode).

Q. Why did classic x86 fail?
A. Some sensitive instructions (e.g. POPF, SGDT) do not trap in user mode.

Q. What does a VM exit mean in VT-x?
A. The CPU switches from the guest (non-root) to the hypervisor (root) because the guest did something the hypervisor must handle.

Q. Which property does an emulator lack?
A. Efficiency.

Go deeper

Read. Popek & Goldberg (1974), CACM 17(7): 412–421. The theorem and the instruction classification.

Read. Robin & Irvine (2000), USENIX Security. The 17 problem x86 instructions.

Read. Barham et al. (2003), *Xen and the Art of Virtualization*, SOSP (in `references/`): para-virtualisation and hypercalls.

Read. Adams & Agesen (2006), ASPLOS. Binary translation vs. hardware assist on x86.

4

The Virtual Private Cloud (VPC)

Chapters 2–3 virtualised the computer. A cloud also has to virtualise the *network*: thousands of tenants share the same switches and cables, yet each must feel it has its own private network. That private network is the **VPC**. (You built one in Lab 3: a VPC with an Nginx server on EC2.)

4.1 Definition

> A **Virtual Private Cloud (VPC)** is a logically isolated virtual network inside a public cloud, in which the customer defines the IP address range, divides it into subnets, controls routing, and applies network security rules to their resources.

A cloud provider is a huge apartment building owned by one landlord. Your VPC is your flat: its own rooms (subnets), its own front door (Internet gateway), its own locks (security groups and network ACLs). The building is shared; the flat is private.

Why we need it: without it, every server would sit in one giant shared network. You must control *who can reach a server, which servers can talk to each other, which can reach the Internet, which must stay private, which ports are open, and which IP addresses are used.* The VPC gives that control.

4.2 The building blocks

1. The CIDR block

When you create a VPC you choose an address range in **CIDR** notation, `a.b.c.d/n`: the first n bits are fixed (the network), the remaining $32 - n$ bits number the hosts, so the block holds 2^{32-n} addresses. VPCs use **private** ranges (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12` and `192.168.0.0/16`. The study notes point out that `10.0.0.0/8` is the usual choice for a large multi-region address plan and `172.16.0.0/12` a common regional default.

2. Subnets

A subnet is a slice of the VPC's range, placed in one **Availability Zone (AZ)**, i.e. one data center of the region.

- **Public subnet**: its route table sends `0.0.0.0/0` to an **Internet Gateway**; used for load balancers, web servers, bastion hosts, NAT gateways.

- **Private subnet**: no route from the Internet; used for application servers and databases.

Watch out

A subnet is not public because you named it "public". It is public because its route table has a route to the Internet Gateway (and its instances have public IPs).

3. Route tables

A route table maps destinations to targets. The packet goes to the **most specific** (longest prefix) matching route.

destination	target	meaning
`10.0.0.0/16`	`local`	anything inside the VPC stays inside
`0.0.0.0/0`	`igw-...`	everything else goes to the Internet (public subnet)
`0.0.0.0/0`	`nat-...`	or to a NAT gateway (private subnet, outbound only)

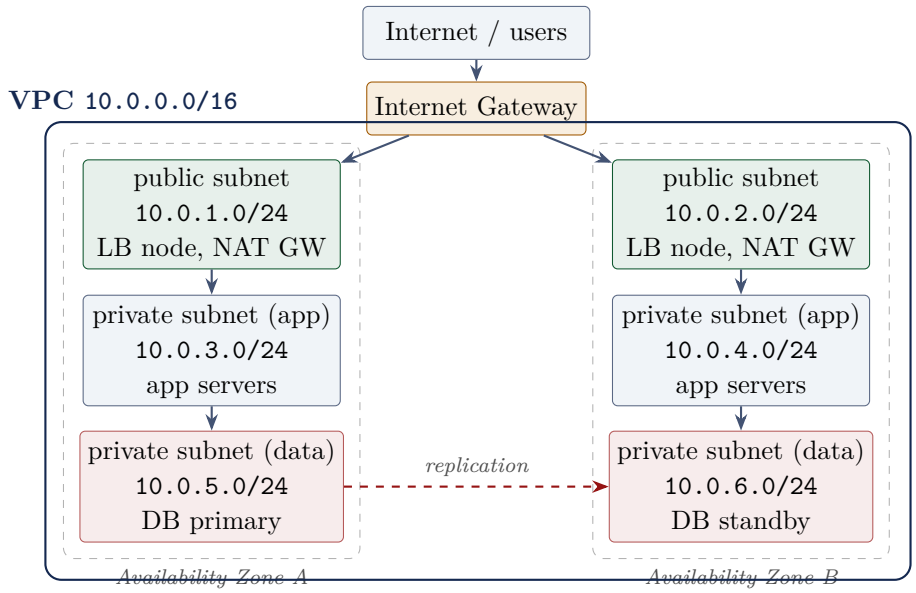
4. Gateways

Internet Gateway (IGW): connects the VPC to the Internet, both directions. **NAT Gateway**: sits in a public subnet and lets private-subnet instances make *outbound* connections (software updates) while blocking inbound ones. **VPC peering / Transit Gateway**: connect VPCs to each other; a transit gateway is a hub that routes between many VPCs and on-premises networks (and can translate addresses when VPC ranges overlap). **VPN / Direct Connect**: link the VPC to an on-premises network (hybrid cloud).

5. Security groups and network ACLs

	Security group	**Network ACL**
Applies to	an instance (its network interface)	a whole subnet
State	**stateful**: reply traffic is allowed automatically	**stateless**: return traffic needs its own rule
Rules	allow only	allow and deny, evaluated in number order
Typical use	"web server accepts 443 from anywhere; DB accepts 3306 only from the app tier's SG"	coarse subnet-wide blocks (e.g. deny a bad IP range)

4.3 The standard picture



A VPC across two AZs: one public and two private subnets in each.

How to draw it in the exam

1. Draw a big rounded box, label it "VPC 10.0.0.0/16" and the region.
2. Split it into two vertical columns: AZ-A and AZ-B (shows high availability).
3. In each column draw three subnet boxes: public (top), private app, private DB; write a /24 in each.
4. Put the Internet Gateway on the VPC border at the top, the Internet above it.
5. Put the load balancer and NAT gateway in the public subnets; app servers in the app subnets; DB primary and standby in the DB subnets with a dashed "replication" arrow.
6. Draw arrows only downward (Internet → LB → app → DB). Write next to the DB: "SG: allow 3306 from app SG only".
7. Add a small route-table box: `10.0.0.0/16 local; 0.0.0.0/0 igw` (public) and `0.0.0.0/0 nat` (private).

4.4 Numericals on addresses

Numerical

Question. (a) How many addresses do `10.0.0.0/16`, `10.0.1.0/24`, `10.0.0.0/20` and `172.16.0.0/12` contain? (b) AWS reserves 5 addresses in every subnet (network, VPC router, DNS, future use, broadcast). How many usable hosts in a /24 and a /20? (c) Split `10.0.0.0/16` into /20 subnets: how many, and what are the first two? (d) Which

addresses are reserved in `10.0.2.0/24`? (e) Is `10.0.2.50` inside `10.0.2.0/24`?
Solution. (a) $2^{32-16} = 65{,}536$; $2^8 = 256$; $2^{12} = 4096$; $2^{20} = 1{,}048{,}576$. (b) /24: $256 - 5 = \mathbf{251}$; /20: $4096 - 5 = \mathbf{4091}$. (c) Going from /16 to /20 borrows 4 bits: $2^4 = \mathbf{16}$ subnets, each 4096 addresses, stepping by 16 in the third octet: `10.0.0.0/20`, `10.0.16.0/20`, …, `10.0.240.0/20`. (d) `10.0.2.0` (network), `.1` (router), `.2` (DNS), `.3` (reserved), `.255` (broadcast). (e) Yes: the first 24 bits are `10.0.2`, the same as the subnet's.

Numerical

Question. Plan the subnets for a three-tier application in `10.0.0.0/16` across two AZs, one /24 per tier per AZ. A packet from an app server is going to `10.0.5.20`; the route table has `10.0.0.0/16 local` and `0.0.0.0/0 nat`. Where does it go?
Solution. Six subnets: web A `10.0.0.0/24`, web B `10.0.1.0/24`, app A `10.0.2.0/24`, app B `10.0.3.0/24`, DB A `10.0.4.0/24`, DB B `10.0.5.0/24` (251 usable each). `10.0.5.20` matches both routes; the longer prefix (/16 vs /0) wins, so it stays **local**: it goes to the DB subnet in AZ-B, never to the NAT gateway.

Check yourself

Q. Define VPC in one line.
A. A logically isolated virtual network in a public cloud where you control IP ranges, subnets, routing and security.

Q. Stateful or stateless: security group?
A. Stateful (NACLs are stateless).

Q. How does a private instance download updates?
A. Through a NAT gateway in a public subnet: outbound only.

Q. What makes a subnet public?
A. A route to the Internet Gateway in its route table.

Part II

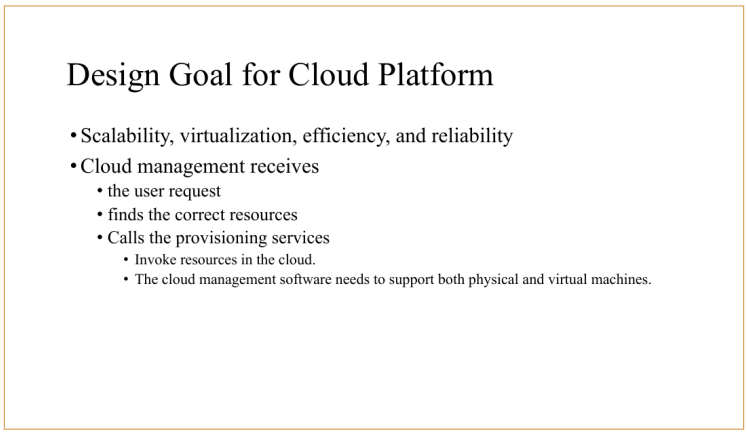
Cloud architecture

5

Cloud Architecture: Design Goals, Layers and Challenges

Week 6 Lecture 1 opens with the provider's view: how a cloud platform itself is organised. Most of these slides (generic security-aware architecture, layered architecture, market-oriented architecture, design challenges) come from Hwang, Fox and Dongarra, *Distributed and Cloud Computing*, Chapter 4. The second half of the lecture (architecture *styles* for applications) is Chapters 6–8.

5.1 Design goals of a cloud platform



Sir's slide (w6-02). Design goals.

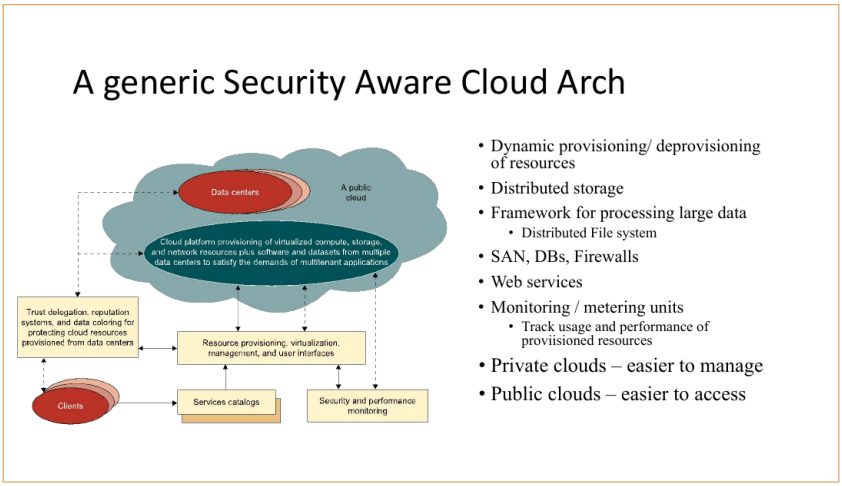
Four goals: **scalability** (add servers and users without redesign), **virtualisation** (everything is a virtual resource that software can create), **efficiency** (high utilisation, low power and cost) and **reliability** (no single failure takes the service down; data is not lost).

The **cloud management** software is the brain. For every user request it

1. receives the request ("give me 3 VMs with 8 GB each"),
2. finds the correct resources (which physical servers have room),
3. calls the **provisioning services**, which invoke (create and start) the resources in the cloud.

It must manage *both* physical machines and virtual machines.

5.2 A generic security-aware cloud architecture



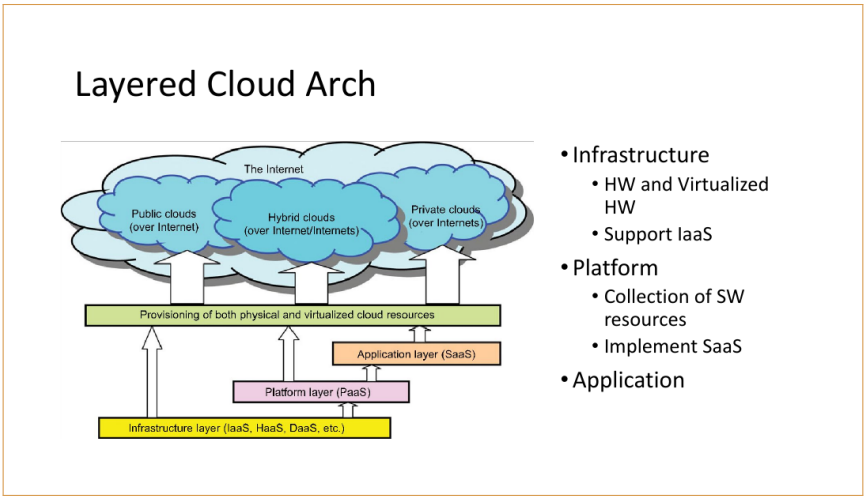
Sir's slide (w6-03). Generic security-aware cloud architecture.

The figure is a public cloud as a set of services that users reach over the Internet. Its parts:

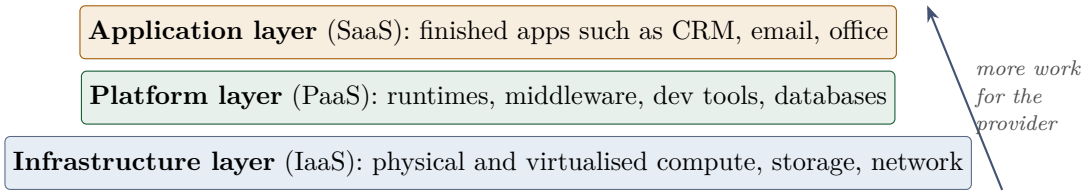
- **Dynamic provisioning and de-provisioning** of resources (VMs come and go with demand).
- **Distributed storage** and a **framework for processing large data** (a distributed file system such as HDFS/GFS with MapReduce-style processing).
- **SANs, databases, firewalls** and **web services** as building blocks.
- **Monitoring and metering units**: track the usage and performance of provisioned resources; this is how you are billed ("measured service") and how SLAs are checked.
- **Security** running through all of it: trust delegation, reputation systems, protection of VMs and of user data.

Sir's two-line comparison: **private clouds are easier to manage**, **public clouds are easier to access**.

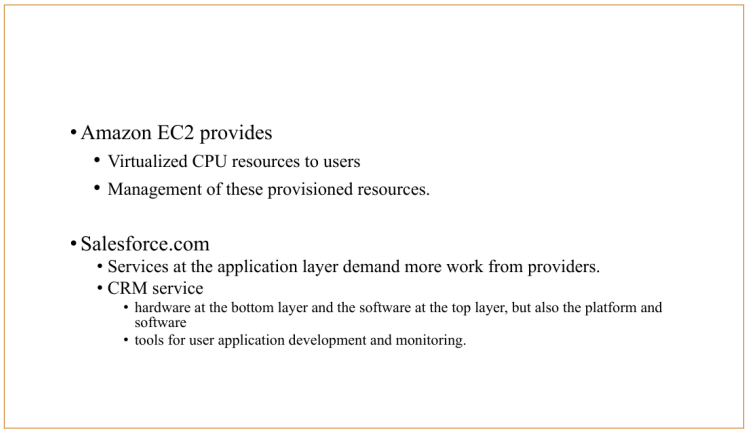
5.3 The layered cloud architecture



Sir's slide (w6-04). Three layers: infrastructure, platform, application.



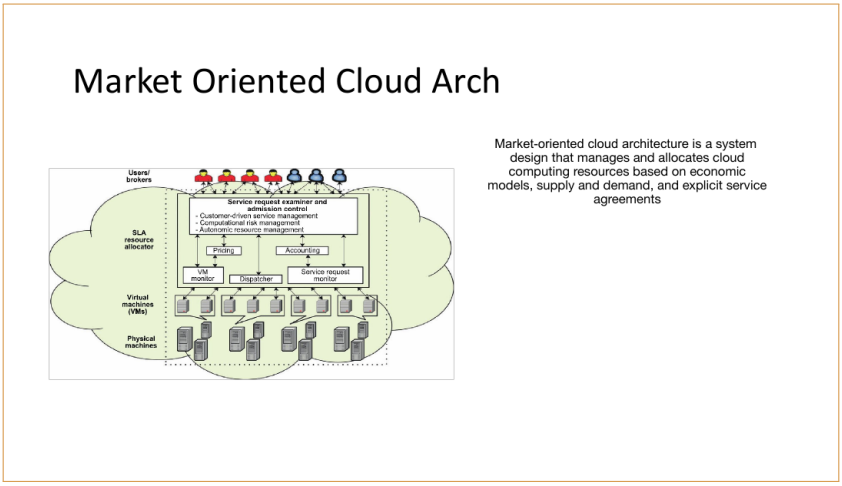
- The **infrastructure layer** is built first, from hardware plus virtualisation; it supports IaaS.
- The **platform layer** is a collection of software resources on top; it is what PaaS sells and what SaaS apps are built on.
- The **application layer** delivers SaaS.



Sir's slide (w6-05). EC2 vs Salesforce.

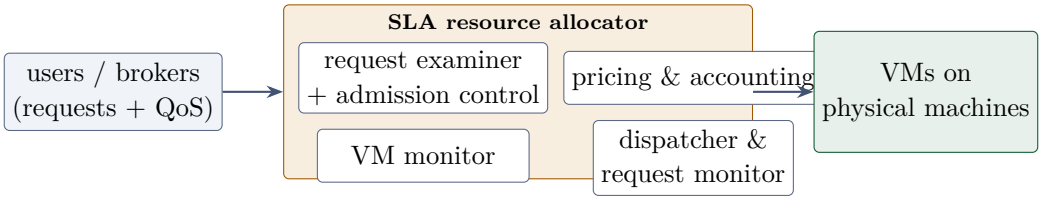
Amazon EC2 works at the bottom: virtualised CPUs plus management of those resources. **Salesforce.com** works at the top: its CRM service means the provider runs *everything*: hardware at the bottom, the platform in the middle, the software at the top, plus tools for users to build and monitor applications. Rule of thumb: *the higher the layer, the more work for the provider.*

5.4 Market-oriented cloud architecture



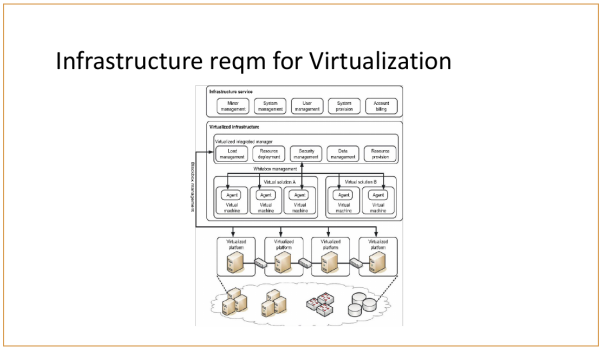
Sir's slide (w6-06). Market-oriented cloud architecture.

> A **market-oriented cloud architecture** allocates resources according to economic models, supply and demand, and explicit service agreements (**SLAs**) with each customer.



Users or brokers submit requests with QoS needs. The **SLA resource allocator** contains: a *service request examiner and admission control* (accept a request only if its SLA can be met), a *pricing* mechanism (price by demand, time, reservation), an *accounting* mechanism (actual usage for billing), a *VM monitor* (which VMs are free and in what state), a *dispatcher* (starts accepted requests on chosen VMs) and a *service request monitor* (tracks progress against the SLA). Underneath are the VMs and physical machines. The point: the cloud behaves like a market that trades resources for money under contracts, not first-come-first-served.

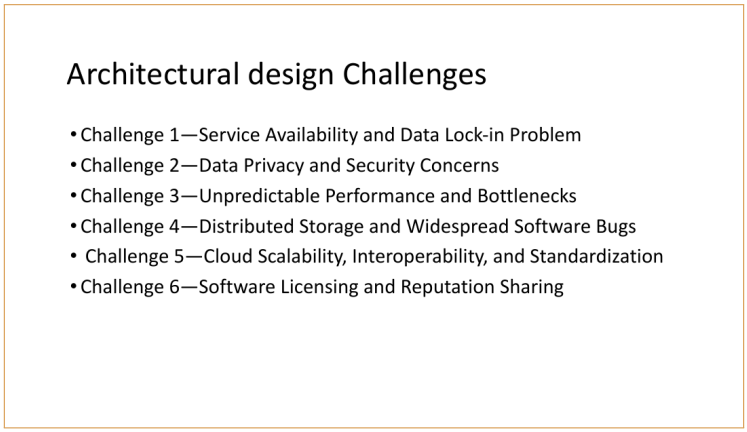
5.5 Infrastructure requirements for virtualisation



Sir's slide (w6-07). Virtualised infrastructure.

From the bottom up: physical servers and storage; a **hypervisor** on every server; **virtual infrastructure management** (VIM) that treats all servers as one pool (create, place, migrate and monitor VMs; examples OpenStack, VMware vCenter); virtual networks and virtual storage; and on top, cloud services and APIs. Combine with Chapter 3: the CPU must meet the Popek–Goldberg condition (or use hardware assist), and memory and I/O must be virtualised.

5.6 Six architectural design challenges



Sir's slide (w6-08). Design challenges.

1. **Service availability and data lock-in.** One provider is a single point of failure: use several regions or providers. Proprietary APIs make data hard to move: standardise APIs.
2. **Data privacy and security.** Data crosses the public Internet and sits on shared hardware: encryption, VPCs, firewalls, and compliance with national data laws.
3. **Unpredictable performance and bottlenecks.** VMs share CPUs well but share I/O and network badly; moving data between sites is slow.
4. **Distributed storage and widespread software bugs.** Storage must grow and shrink on demand; some bugs appear only at scale and cannot be reproduced on one machine, so debugging happens in the cloud.
5. **Scalability, interoperability and standardisation.** Pay-as-you-go must scale quickly up and down; formats such as OVF (Open Virtualization Format) let VMs move between hypervisors.
6. **Software licensing and reputation sharing.** Per-machine licences fit clouds badly; one tenant's misbehaviour (spam, attacks) can get the provider's IP addresses blacklisted for everyone.

Check yourself

Q. Four design goals of a cloud platform?
A. Scalability, virtualisation, efficiency, reliability.

Q. The three layers and the service model each supports?
A. Infrastructure–IaaS, platform–PaaS, application–SaaS.

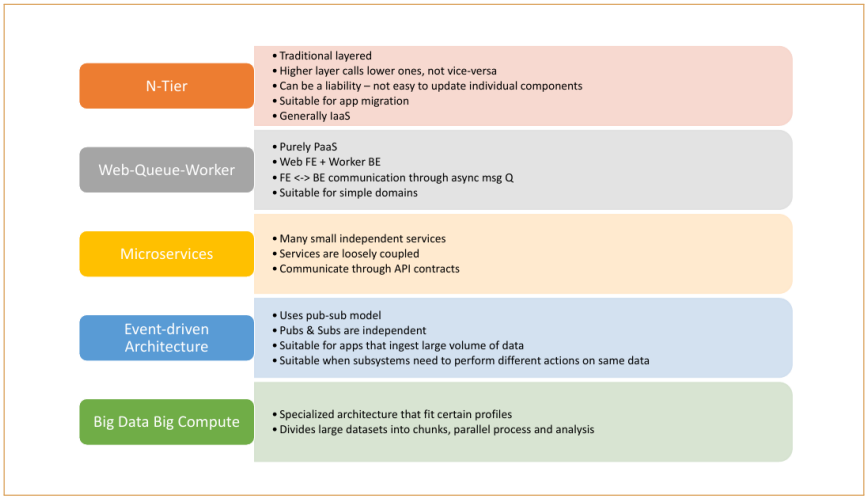
Q. What does the SLA resource allocator do?
A. Admission control, pricing, accounting, VM monitoring and dispatch of requests according to SLAs.

Q. Name the six design challenges.
A. Availability & lock-in; privacy & security; unpredictable performance; distributed storage & bugs; scalability & standardisation; licensing & reputation.

6

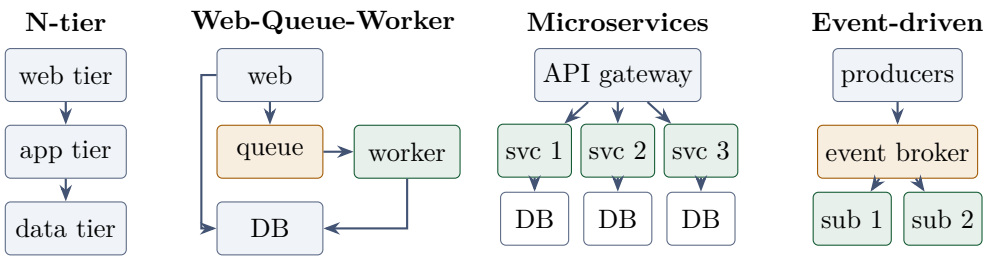
Types of Cloud Architecture (Architecture Styles)

An **architecture style** is a family of designs for a cloud *application* with shared characteristics. Sir's slide 9 lists five, following the Azure Architecture Center. The next two chapters go deep into the two on the syllabus (N-tier and Web-Queue-Worker); this chapter gives all five side by side, because "compare" and "which style would you choose" questions are common.



Sir's slide (w6-09). Five architecture styles.

6.1 The five styles in one picture each



6.2 Comparison

Style	Shape	Good for	Weakness
N-tier	horizontal layers (presentation, logic, data); a higher layer calls lower ones, never the reverse	traditional apps, migrating on-premises apps ("lift and shift"); generally **IaaS**	monolithic: hard to update one part independently
Web-Queue-Worker	web front end + worker back end, joined by an **asynchronous message queue**	simple domains with long-running or batch jobs; purely **PaaS**	web and worker can grow into monoliths; hidden coupling via shared schema
Microservices	many small independent services, each owning its data, talking through **API contracts**	large complex domains, many teams, frequent independent releases	distributed-systems complexity: networking, data consistency, monitoring
Event-driven	**publish–subscribe**: producers emit events, independent subscribers react	ingesting large volumes of data (IoT), several subsystems doing different things with the same data	ordering, exactly-once delivery, harder to reason about
Big data / big compute	split a huge dataset or computation into chunks, process in parallel, combine	analytics, simulation, rendering, ML training	specialised; only fits certain workloads

How to choose in an exam answer

Ask four questions. *Is it a traditional app or a migration?* N-tier. *Is the domain simple but some jobs are slow (video encoding, report generation)?* Web-Queue-Worker. *Is the domain big, with many teams and independent release cycles?* Microservices. *Is the core of the system a stream of events that several consumers react to?* Event-driven. Then justify with scalability, cost, complexity.

Check yourself

Q. Which style is "purely PaaS"?

A. Web-Queue-Worker (sir's slide); N-tier is generally IaaS.

Q. How do microservices communicate?

A. Through well-defined API contracts; event-driven systems use publish–subscribe.

Q. A higher layer calls a lower one, never the reverse. Which style?

A. N-tier (layered).

Q. Why is N-tier called a liability for updates?

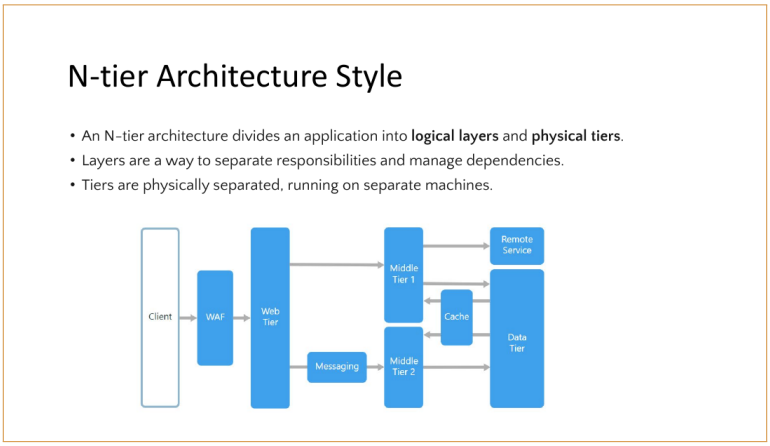
A. Layers are tightly coupled and usually deployed together, so updating one component often means redeploying the tier.

7

N-tier Architecture

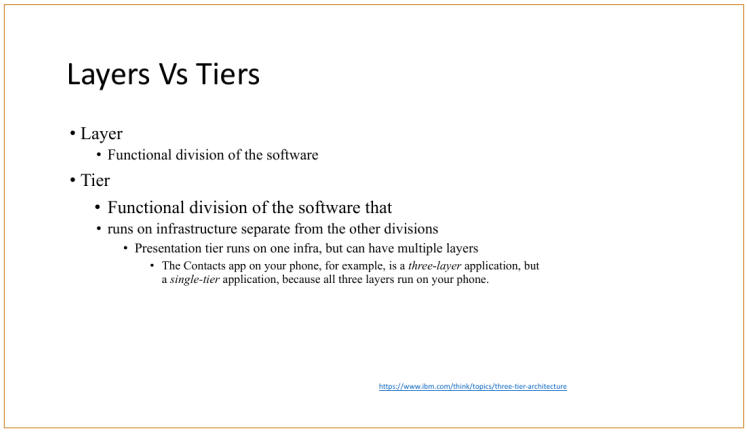
This is the chapter to know best. It covers sir's slides 10–17, the Azure Architecture Center's N-tier guide, and the AWS three-tier reference. It ends with how to *draw* an N-tier design on paper and four fully worked design questions, because "design an N-tier architecture for …" is the most likely long question.

7.1 Definition: layers and tiers

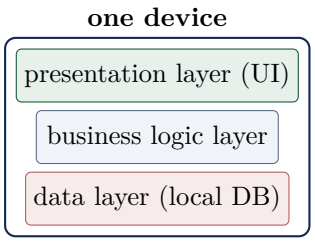


Sir's slide (w6-10). N-tier architecture style.

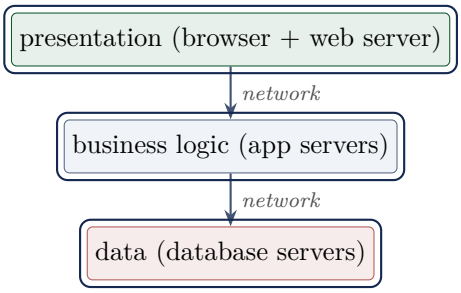
> An **N-tier architecture** divides an application into **logical layers** and **physical tiers**.
>
> - A **layer** is a *functional* division of the software (presentation, business logic, data access). Layers separate responsibilities and manage dependencies: a higher layer may use a lower one, never the reverse.
> - A **tier** is a layer (or group of layers) that runs on *separate infrastructure* (its own machines or VMs). Tiers talk to each other over the network.



Sir's slide (w6-16). Layers vs tiers.



Contacts app on a phone:
3 layers, 1 tier



web application:
3 layers, 3 tiers

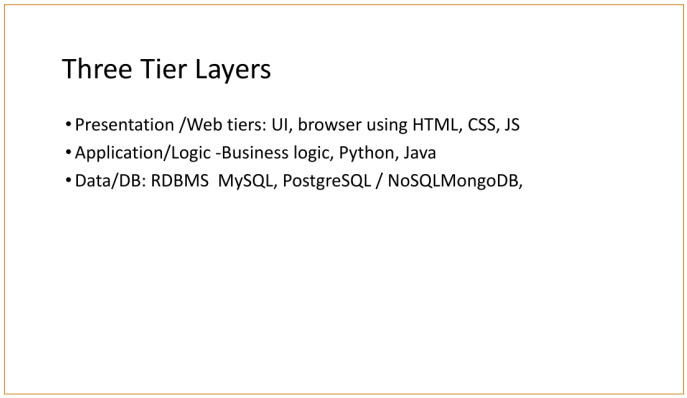
Watch out

"Layer" and "tier" are not synonyms. A tier is *physical*: if two layers run on the same machine they are one tier. Your Library assignment had presentation, business and data layers in one Python process: three layers, one tier. Deploy the database on its own server and it becomes two tiers.

Closed vs open layers; strict vs relaxed tiers

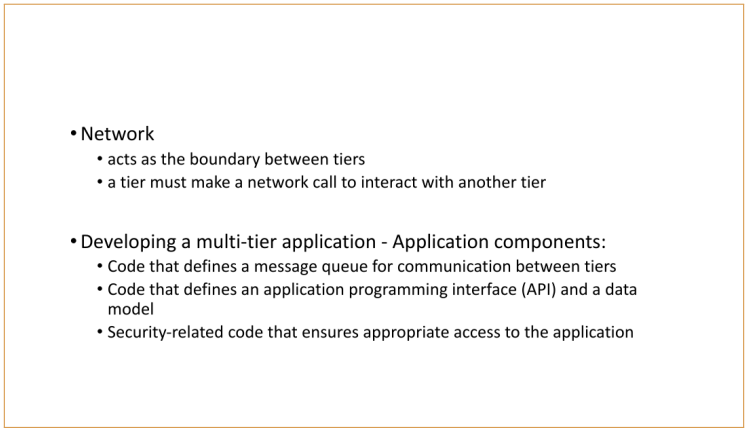
- **Closed layer architecture**: a layer may call only the layer *immediately* below it. Fewer dependencies, but a layer that just passes requests through adds traffic and latency.
- **Open layer architecture**: a layer may call *any* layer below it.
- For tiers, the same idea is called **strict** (a request must pass through every adjacent tier; more latency) vs **relaxed** (requests may skip tiers; more coupling). Both can be mixed.

7.2 The three-tier architecture



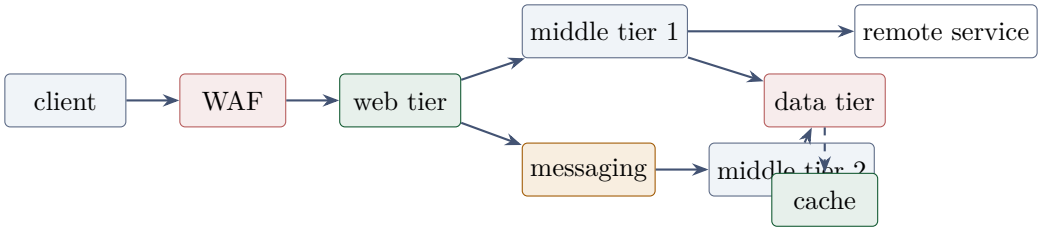
Sir's slide (w6-14). Three-tier layers.

1. **Presentation / web tier**: the user interface. Browser with HTML, CSS, JavaScript; web servers (Apache, Nginx) serving pages and static content.
2. **Application / logic tier**: business rules in Python, Java, Node.js, .NET: validate an order, compute a price, check stock.
3. **Data tier**: storage. Relational (MySQL, PostgreSQL, Amazon RDS) or NoSQL (MongoDB, DynamoDB).



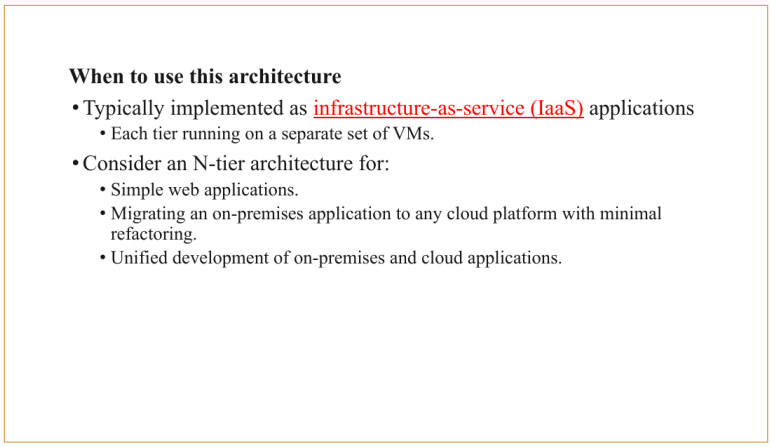
Sir's slide (w6-15). The network as the boundary; components of a multi-tier app.

The **network is the boundary between tiers**: a tier must make a network call to use another tier. Building a multi-tier application therefore needs three kinds of code besides the business logic: (1) code that defines a **message queue** or other channel for communication between tiers; (2) code that defines an **API and a data model** (what a request and a response look like); (3) **security** code that ensures only the right callers reach each tier.



The Azure logical N-tier diagram: more than three tiers, a queue between web and one middle tier, a cache by the data.

7.3 When to use N-tier, benefits, challenges



Sir's slide (w6-11). When to use this architecture.

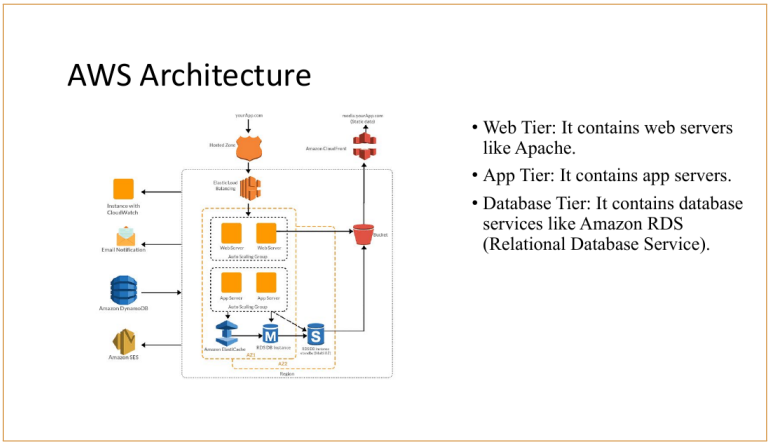
Typically implemented as **IaaS**, each tier on its own set of VMs. Use it for: simple web applications; **migrating an on-premises application to the cloud with minimal refactoring**; unified development of on-premises and cloud applications; requirements that are still evolving.

Benefits	Challenges
portable between on-premises and clouds	a middle tier that only does CRUD adds latency for nothing
small learning curve for developers	monolithic: features cannot be deployed independently
cheap: no re-architecture needed	network security is hard to manage in large systems
natural evolution of the traditional model	requests crossing many tiers are harder to test and monitor
each tier scales independently	extra network hops add latency

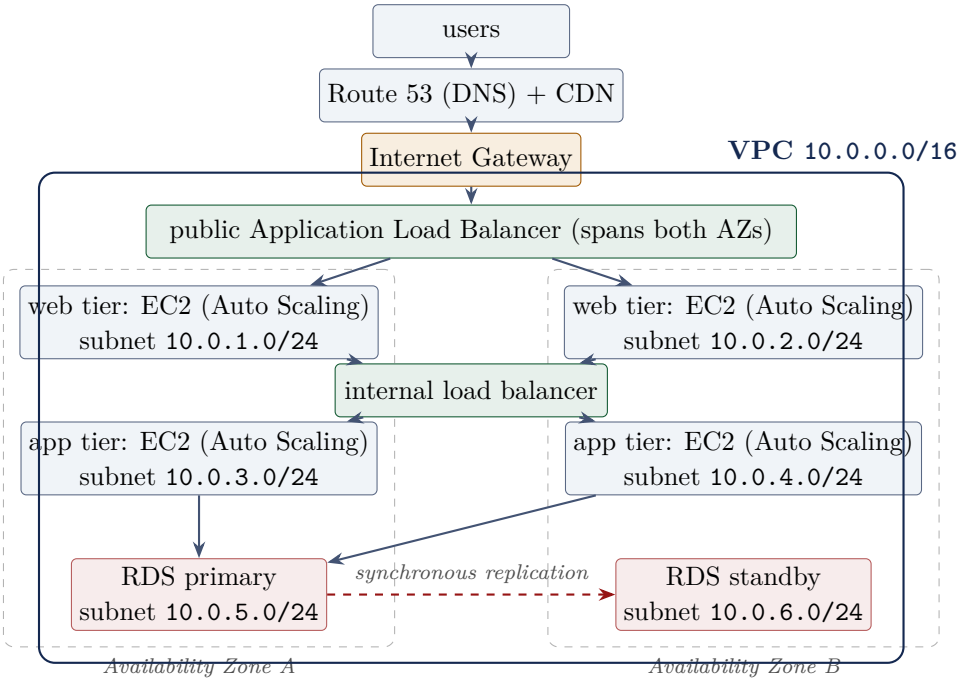
Best practices (Azure): autoscale each tier; use asynchronous messaging to decouple tiers; cache data that rarely changes; make the data tier highly available (replication, e.g. SQL Server Always On); put a **web application firewall (WAF)** between the Internet and the front end; put **each tier in its own subnet** and use subnets as security boundaries; **let only the middle tier reach the data tier**.

7.4 Reference designs

AWS three-tier



Sir's slide (w6-12). AWS architecture: web tier, app tier, database tier (RDS).



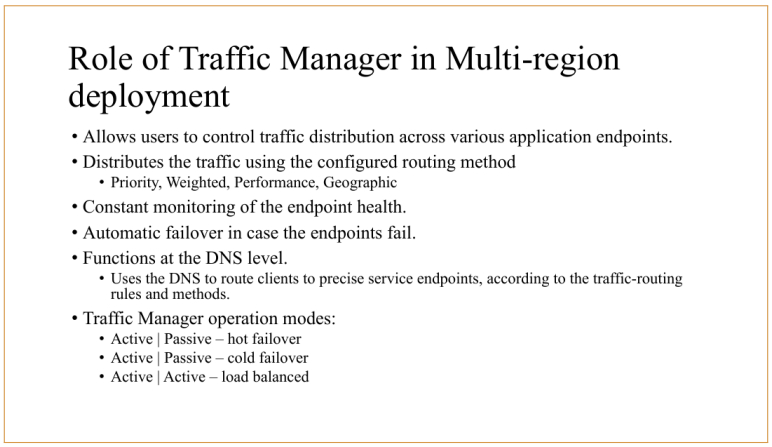
A highly available AWS three-tier design. Draw this one in the exam.

Reading it from the top: users resolve the name through **Route 53**; static files come from a **CDN** (CloudFront); traffic enters through the **Internet Gateway** to a public **Application Load Balancer**, which spreads requests over web servers in **two Availability Zones**; each tier is an **Auto Scaling group** that adds or removes EC2 instances with load; an **internal load balancer** spreads web-to-app calls; the app tier reads and writes **Amazon RDS**, whose primary in AZ-A replicates synchronously to a standby in AZ-B (**Multi-AZ**); if AZ-A fails, RDS fails over to the standby automatically. Security groups: ALB accepts 443 from anywhere; web accepts only from the ALB's SG; app only from the web SG; DB only port 3306/5432 from the app SG. Admins reach instances through a bastion host or Session Manager, never directly.

Azure N-tier on virtual machines

Each tier is a **virtual machine scale set** of two or more VMs behind a load balancer, in its own subnet with a network security group (the database tier accepts traffic only from the business tier). The web and business tiers are **stateless** (any VM can serve any request); the data tier is replicated (SQL Server Always On, or Cassandra on Linux). In front sits a **perimeter network (DMZ)** with network virtual appliances (firewalls) behind an external load balancer; admins use **Azure Bastion**; the VNet can be linked to on-premises by VPN or ExpressRoute.

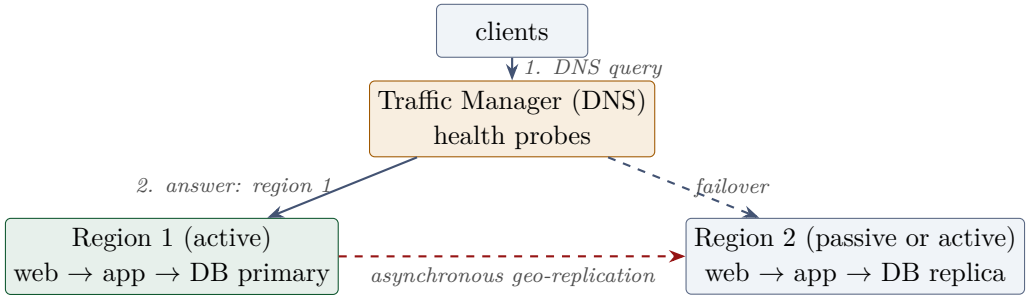
7.5 Multi-region deployment and Traffic Manager



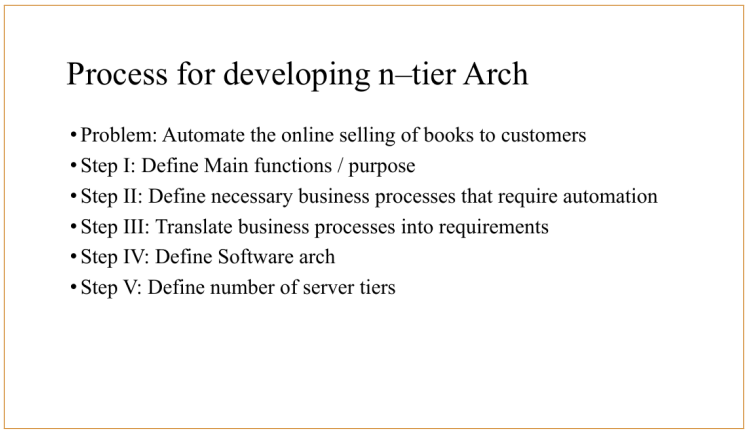
Sir's slide (w6-13). Role of Traffic Manager in multi-region deployment.

A single region can still fail. For higher availability, deploy the whole N-tier stack in two regions and put **Azure Traffic Manager** (AWS equivalent: Route 53 routing policies) in front.

- It works at the **DNS level**: it answers the client's DNS query with the address of the chosen endpoint; traffic then flows directly to that endpoint, not through Traffic Manager.
- **Routing methods**: **priority** (always the primary unless it is down), **weighted** (split by weights, e.g. 3:1), **performance** (lowest-latency region for the user), **geographic** (by the user's location, e.g. for data-residency laws).
- It **monitors endpoint health** continuously and **fails over automatically**.
- **Modes**: *active–passive, hot failover* (standby region running and ready), *active–passive, cold failover* (standby region created or started only on failure: cheaper, slower), *active–active* (both regions serve traffic, load-balanced).



7.6 How to develop an N-tier architecture: sir's five steps



Sir's slide (w6-17). Process for developing an N-tier architecture.

Worked on sir's own problem: *automate the online selling of books to customers.*

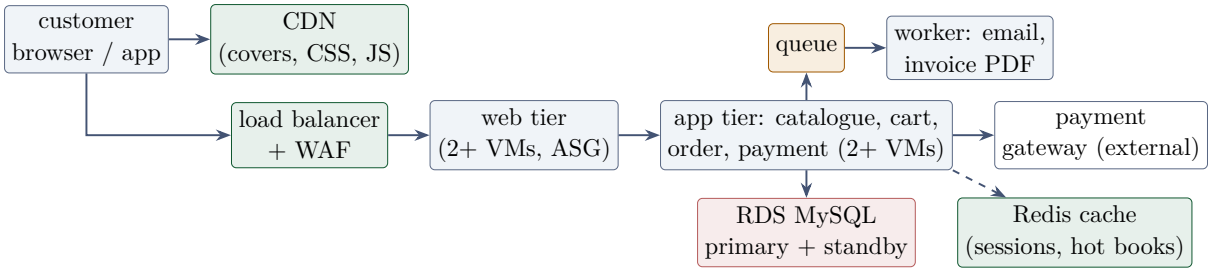
Step I: define the main functions / purpose.
Customers browse and search books, add to cart, pay, and track orders; staff manage the catalogue, stock and shipments.

Step II: define the business processes that need automation.
(1) catalogue browsing and search; (2) customer registration and login; (3) cart and checkout; (4) payment; (5) order fulfilment and shipping; (6) inventory update; (7) notifications (email/SMS); (8) reports for managers.

Step III: translate processes into requirements.
Functional: search by title/author/ISBN, secure login, cart persists across sessions, card/UPI payment via a gateway, order status page, stock decremented on purchase, email confirmation. Non-functional: 99.9% availability, under 2 s page load, handle 10× traffic during sales, PCI-DSS for payments, encrypted personal data, daily backups.

Step IV: define the software architecture.
Presentation (web UI and mobile app talking to a REST API), business logic (catalogue, cart, order, payment, inventory, notification modules), data access (repositories), data (relational DB for orders, customers and stock; search index for the catalogue; object storage for cover images; cache for hot books and sessions).

Step V: define the number of server tiers.
Four tiers: (1) web tier (load balancer + web servers + CDN), (2) application tier (API servers), (3) data tier (RDS primary + standby, read replica), plus (4) an asynchronous worker tier fed by a queue for emails and invoice PDFs (a Web-Queue-Worker element inside an N-tier design).



The bookstore after Step V.

7.7 Drawing an N-tier architecture on paper

How to draw it in the exam

Logical diagram (draw first, 2 minutes).

1. Three (or more) horizontal boxes stacked top to bottom: Presentation, Business logic, Data. Write the technology inside each (HTML/JS + Nginx; Java/Python; MySQL).
2. Arrows only *downward*; label them "HTTPS", "REST/gRPC", "SQL".
3. Optional boxes to the side: cache next to the data tier, queue between web and a worker.

Physical (deployment) diagram (5 minutes).

1. Outer box: cloud region; inside it a VPC box with its CIDR.
2. Two vertical columns: AZ-A and AZ-B.
3. Rows: public subnet (load balancer, NAT), private app subnet, private DB subnet. One /24 per box.
4. Top: users → DNS → Internet Gateway → load balancer spanning both AZs.
5. Draw two or more server icons per tier and write "Auto Scaling group".
6. DB: primary in AZ-A, standby in AZ-B, dashed arrow "sync replication"; optionally a read replica.
7. Security: write the security-group rule beside each tier ("allow 80/443 from LB", "allow 8080 from web SG", "allow 3306 from app SG").
8. Extras that earn marks: WAF, CDN for static content, cache, bastion host, monitoring (CloudWatch), backups.

Then write five lines of justification: why each tier is separate (independent scaling, security boundary), how it scales (ASG + LB), how it survives failure (2 AZs, Multi-AZ DB), how it is secured (private subnets, SG chaining, WAF), what is cached.

7.8 Worked design questions

Design 1: college result portal

Question. Design an N-tier architecture for a university results portal: 20,000 students check results; traffic is near zero most of the year and very high for two days after results are declared. Faculty upload marks through a separate admin interface.

Answer outline. *Tiers*: (1) web tier: static result page from a CDN + 2–20 web VMs in an Auto Scaling group behind an ALB; (2) app tier: results API, admin upload API (separate target group, reachable only from the campus VPN or with SSO); (3) data tier: relational DB (results table indexed by roll number) with a **read replica**, since the spike is almost all reads. *Spike handling*: **scheduled scaling** before the declaration time; cache each student's result in Redis after the first read; the CDN absorbs the page assets. *Security*: admin endpoints in a private subnet; DB reachable only from the app SG; marks upload logged. *Why N-tier*: simple domain, read-heavy, needs independent scaling of the web tier only.

Design 2: food delivery application

Answer outline. Presentation: customer app, restaurant app, rider app, all calling an API through an API gateway / ALB. Logic tier: order, menu, payment, rider assignment modules. Data tier: relational DB for orders and payments, NoSQL for menus, Redis for rider locations. Asynchronous part: order events to a queue; workers send notifications and assign riders (Web-Queue-Worker inside N-tier). Multi-AZ everywhere; Traffic Manager / Route 53 latency routing if the service spans countries.

Design 3: migrating an on-premises banking application

Answer outline. N-tier is the right choice precisely because it is a **migration with minimal refactoring**: lift the existing web, app and DB servers onto VMs (IaaS), one tier per subnet. Put a perimeter network with firewalls (NVAs) and a WAF in front; Bastion for admin access; site-to-site VPN or Direct Connect/ExpressRoute back to the bank's data center (hybrid cloud); regulated customer data may stay on-premises ("control when needed"). Data tier: synchronous replication across AZs (RPO near zero), asynchronous to a second region for disaster recovery with Traffic Manager priority routing (active–passive, hot).

7.9 Numericals

Numerical

Question (availability). A three-tier app has a load balancer (99.99%), web servers (99% each), app servers (99% each) and DB servers (99.9% each). (a) What is the availability with one server per tier? (b) With 2 web, 3 app and 2 DB servers (any one in a tier suffices)? Express downtime per year.

Solution. Components in **series** (all needed) multiply: $A = \prod A_i$. Identical components in **parallel** (any one enough) fail only if all fail: $A = 1 - (1 - a)^n$.

(a) $0.9999 \times 0.99 \times 0.99 \times 0.999 = 0.97902$. Downtime $= (1 - 0.97902) \times 8760 \approx$ **184** h/yr

(7.7 days).

(b) Web $1-0.01^2 = 0.9999$; app $1-0.01^3 = 0.999999$; DB $1-0.001^2 = 0.999999$. Total $= 0.9999 \times 0.9999 \times 0.999999 \times 0.999999 \approx 0.999798$. Downtime $\approx$ **106** min/yr.

Redundancy turns a week of downtime into under two hours; the weakest link is now the single load balancer and the two-server web tier. (Reference: 99.9% $\approx$ 8.76 h/yr; 99.99% $\approx$ 52.6 min/yr; 99.999% $\approx$ 5.3 min/yr.)

Numerical

Question (capacity). Peak load is 6000 requests/s. A web VM handles 800 req/s and an app VM 400 req/s; keep each VM at most 70% busy. How many VMs per tier? How many if the tier must survive the loss of one AZ out of two?
Solution. Web: $\lceil 6000/(800 \times 0.7) \rceil = \lceil 10.7 \rceil =$ **11**. App: $\lceil 6000/(400 \times 0.7) \rceil = \lceil 21.4 \rceil =$ **22**. To survive an AZ failure, *each* AZ must carry the full load alone: 11 web and 22 app VMs *per AZ* (22 and 44 in total). A cheaper compromise is to run $N/2+1$ per AZ and let auto scaling add the rest after a failure (6 and 12 per AZ), accepting a few minutes of overload.

Numerical

Question (latency and caching). Each network hop between tiers costs 2 ms and there are 3 hops (client→web, web→app, app→DB). Processing: web 5 ms, app 20 ms, DB query 15 ms. A cache answers 80% of DB reads in 1 ms. Find the response time without and with the cache.
Solution. Without cache: $3 \times 2 + 5 + 20 + 15 =$ **46** ms. With cache, expected data time $= 0.8 \times 1 + 0.2 \times 15 = 3.8$ ms, so $6 + 5 + 20 + 3.8 =$ **34.8** ms. The extra tiers cost 6 ms of network; caching recovers more than that.

Numerical

Question (Traffic Manager). Two regions get weights 3 and 1. What share of users goes to each? Health probes run every 10 s and an endpoint is marked down after 3 consecutive failures; the DNS TTL is 60 s. What is the worst-case time before all users reach the healthy region?
Solution. Shares: $3/(3+1) =$ **75**% and **25**%. Detection takes up to $3 \times 10 = 30$ s; clients may then keep a cached DNS answer for up to the 60 s TTL: worst case $\approx 30 + 60 =$ **90** s. This is why DNS-based failover uses short TTLs.

Check yourself

Q. Layer vs tier in one line each.
A. Layer: functional division of software. Tier: a layer running on separate infrastructure.

Q. Why place each tier in its own subnet?
A. So subnets act as security boundaries: NSG/SG rules per tier, DB reachable only from the app tier.

Q. Four Traffic Manager routing methods?
A. Priority, weighted, performance, geographic.

Q. At what level does Traffic Manager work?

A. DNS.

Q. Hot vs cold failover?

A. Hot: standby running and ready. Cold: standby started only after failure (cheaper, slower).

Q. When is N-tier the right choice?

A. Simple web apps and migrating on-premises apps with minimal refactoring; generally IaaS.

8

The Web-Queue-Worker Architecture

Sir's slide 9 describes Web-Queue-Worker in four lines: *purely PaaS; web front end + worker back end; front end and back end communicate through an asynchronous message queue; suitable for simple domains.* This chapter explains each line, draws the architecture, compares it with N-tier, and does the queue numericals that make it concrete.

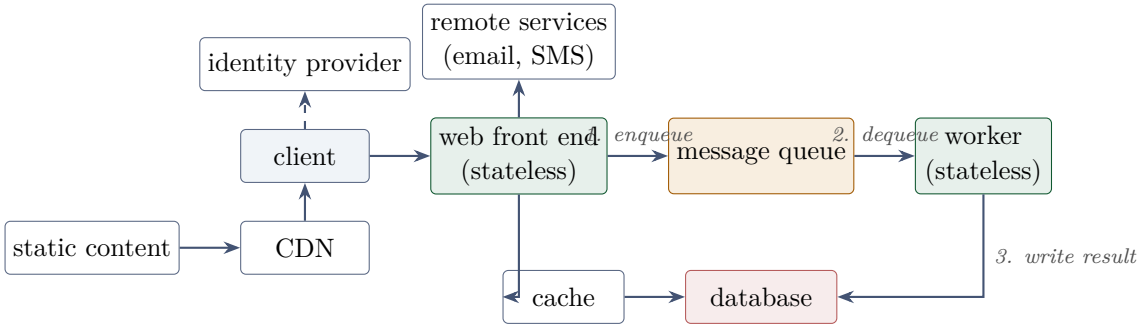
8.1 The problem it solves

> **Ask before reading on**
>
> A user uploads a 2-minute video that takes 40 s to transcode. If the web server transcodes it inside the HTTP request, what happens to the user, and to the web server, when 200 people upload at once?

The user waits 40 s staring at a spinner (and the request may time out); each web server thread is blocked for 40 s, so a few hundred uploads exhaust the web tier and even simple page views start failing. The fix: the web server only *records the job* and replies immediately; a separate **worker** does the heavy work later. Something must hold the jobs in between: a **queue**.

8.2 The architecture

> **Web-Queue-Worker** has two core components: a **web front end** that handles client requests, and a **worker** that performs resource-intensive tasks, long-running workflows or batch jobs. The web front end communicates with the worker through a **message queue**.



Logical view (after the Azure Architecture Center diagram).

Typical supporting pieces: one or more **databases**; a **cache** for fast reads; a **CDN** for static content; **remote services** such as email or SMS; an **identity provider**. Both web and worker are **stateless** (session state goes into a distributed cache), so either can scale out. The worker can be triggered by queue messages or run on a schedule for batch work, and it is optional if there are no long-running operations. The front end may expose a web API consumed by a single-page or mobile app.

One request, step by step (video upload)

1. The browser uploads the file to blob storage and posts "transcode video 42" to the web front end.
2. The web front end writes a row (status = *queued*) to the database, puts a message `{video: 42}` on the queue, and replies `202 Accepted` in about 50 ms.
3. A worker takes the message off the queue, transcodes for 40 s, writes the output, sets status = *done*, and deletes the message. If the worker crashes, the message becomes visible again and another worker retries it.
4. The page polls (or gets a push notification) and shows the finished video.

8.3 Why "purely PaaS"

It is normally built from managed platform services: the web front end on **Azure App Service** (or AWS Elastic Beanstalk / ECS), the worker as **Azure Functions** (or AWS Lambda), the queue as **Azure Service Bus / Storage queues** (or Amazon SQS), plus managed Redis, SQL Database / Cosmos DB and a CDN. You deploy code; you do not manage VMs. Contrast N-tier, which is generally IaaS (one set of VMs per tier).

8.4 When to use, benefits, challenges

Use it for: a relatively **simple domain**; applications with **long-running workflows or batch operations**; teams that prefer managed services over IaaS.

Benefits	**Challenges**
simple, easy to understand	web and worker can each grow into a large monolith
easy deployment and management	shared data schemas or code create hidden coupling
clean separation of front end and worker	the front end can write to the DB and then crash before sending the message: the job is never processed (fix: **transactional outbox**)
asynchronous messaging decouples them	jobs complete *later*: the UI must show progress
web and worker **scale independently**	the queue needs monitoring (length, age, poison messages)

Best practices: a well-designed API; autoscaling (schedule-based if load is predictable, metric-based, e.g. on queue length, if not); cache semi-static data; serve static content from a CDN; **polyglot persistence** (different stores for different data); partition data; put web and worker in separate App Service plans so they scale independently; deployment slots for zero-downtime releases.

The queue is a shock absorber

The queue lets the worker run at its own steady pace while arrivals come in bursts. This is the *queue-based load levelling* pattern: the front end never waits for the slow part, and a burst becomes a backlog that drains, instead of a crash.

8.5 Web-Queue-Worker vs N-tier

	N-tier	**Web-Queue-Worker**
Hosting	generally IaaS (VMs per tier)	purely PaaS (managed services)
Communication	mostly synchronous calls down the layers	asynchronous through a queue
Long jobs	block a request thread	handled by the worker in the background
Best for	traditional apps, migrations	simple domains with heavy background work
Scaling unit	a tier	the web front end and the worker separately

They also combine: the bookstore of Chapter 7 uses a queue and worker for emails and invoices.

How to draw it in the exam

1. Left to right: Client → Web front end → Queue (draw it as a pipe or a stack of envelopes) → Worker.
2. Below the middle: Database; both web and worker connect to it. Put a Cache between web and DB.
3. Above the client: CDN with static content; identity provider.
4. Number the arrows 1–4 (request, enqueue, dequeue and process, write result) and write "`202 Accepted` immediately" on the reply arrow.
5. Write "stateless, autoscale on queue length" under the worker, "PaaS: App Service / Functions / Service Bus" as a caption.

8.6 Numericals

Numerical

Question (how many workers?). Jobs arrive at 20 per minute; each takes a worker 45 s. How many workers are busy on average? What is the minimum number of workers for the queue not to grow forever? How many for an 80% utilisation target?
Solution. Arrival rate $\lambda = 20/60 = \frac{1}{3}$ job/s. Offered load $= \lambda \times S = \frac{1}{3} \times 45 =$ **15** workers busy on average. With exactly 15 workers utilisation would be 100% and any burst would grow the queue without bound, so the minimum stable number is **16**. For 80%: $\lceil 15/0.8 \rceil =$ **19**.

Numerical

Question (burst and drain). A promotion causes 10 uploads/s for 10 minutes. There are 25 workers, each finishing a job every 4 s. How big is the backlog at the end of the burst, how long does it take to drain, and how many workers would keep up?
Solution. Capacity $= 25 \times \frac{1}{4} = 6.25$ jobs/s. The queue grows at $10 - 6.25 = 3.75$ jobs/s for 600 s: backlog = **2250** jobs. After the burst it drains at 6.25/s: $2250/6.25 =$ **360** s (6 min); the last job of the burst waits about 6 minutes. To keep up, need $10/0.25 = 40$ workers, i.e. 15 more: autoscaling on queue length would add them. Note the web tier never slowed down: users got `202 Accepted` the whole time.

Numerical

Question (Little's law). A queue receives 0.5 jobs/s and each job spends 120 s in the system (waiting + processing). How many jobs are in the system on average?
Solution. Little's law $L = \lambda W = 0.5 \times 120 =$ **60** jobs. It holds for any stable queue, whatever the arrival pattern: a handy check on monitoring dashboards.

Numerical

Question (synchronous vs asynchronous). Transcoding takes 40 s. Each web server can hold 100 concurrent requests. In the synchronous design, how many uploads per second can one web server accept? In Web-Queue-Worker, where the web server spends 50 ms per upload request?
Solution. Synchronous: each request occupies a slot for 40 s, so throughput $= 100/40 =$ **2.5** uploads/s per server. Asynchronous: $100/0.05 =$ **2000** requests/s per server: 800 times more, because the slow work moved to the workers.

Check yourself

Q. Two core components and the connector?
A. Web front end and worker, connected by a message queue.

Q. Why must the web front end and worker be stateless?
A. So any instance can handle any request or job and both can scale out; state lives in the DB or a distributed cache.

Q. What problem does the transactional outbox solve?
A. The front end writing the DB row but crashing before sending the queue message.

Q. Give two good use cases.

A. Video/image processing, report or invoice generation, sending bulk email, ML inference batches.

Part III

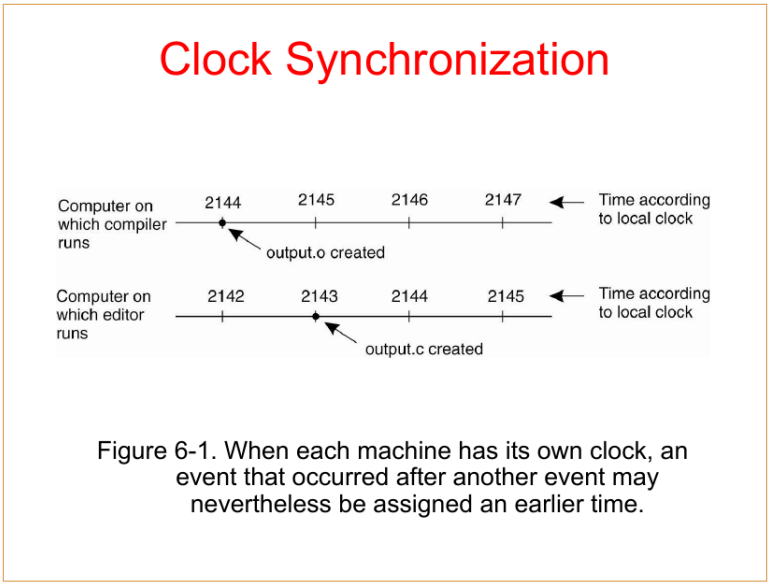
Distributed systems: time

9

Time in Distributed Systems: Physical Clocks and Synchronisation

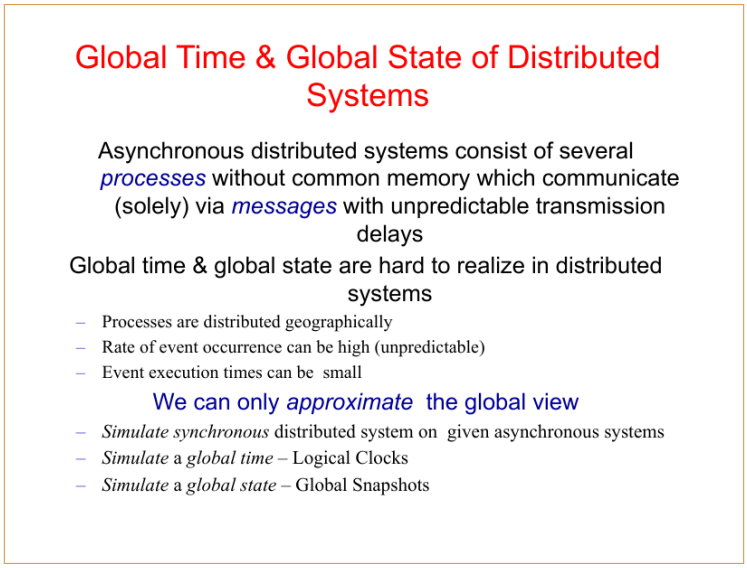
Week 8 Lecture 2 (Tanenbaum and van Steen, *Distributed Systems*, Chapter 6). A distributed system has no shared memory and no shared clock: processes cooperate only by sending messages whose delays are unpredictable. This chapter is about keeping *physical* clocks close together; the next is about ordering events without trusting physical clocks at all.

9.1 Why clocks matter



Sir's slide (w8-02). Figure 6-1: when each machine has its own clock.

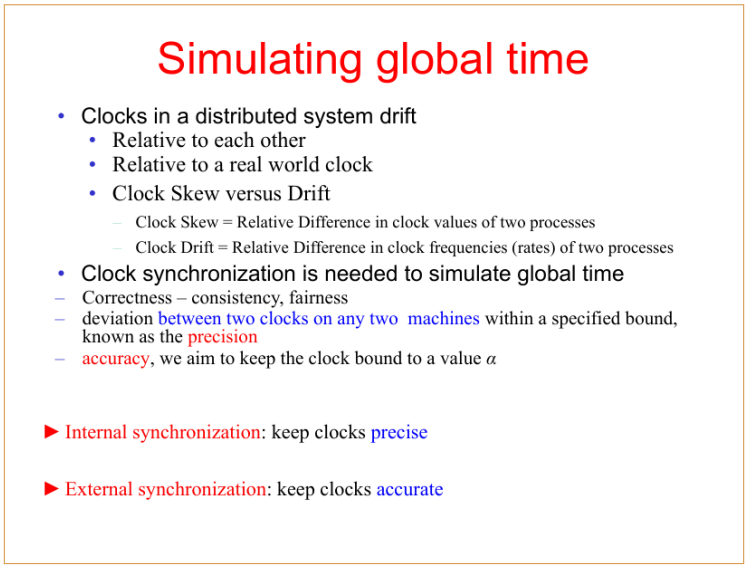
`make` recompiles a file when the source is newer than the object file. The editor runs on a machine whose clock is behind the compiler's machine. `output.o` is created at 2144 on the compiler machine; `output.c` is then edited at 2143 on the editor machine. Although the edit happened *after* the compilation, it carries an *earlier* time, so `make` thinks the object file is up to date and does not recompile. A real bug caused purely by clocks disagreeing.



Sir's slide (w8-03). Global time and global state.

Global time and global state are hard because processes are geographically spread, events happen at high and unpredictable rates, and events are short. We can only *approximate* a global view: simulate global time with **logical clocks** (Chapter 10) and simulate a global state with **global snapshots**.

9.2 Skew, drift, precision, accuracy

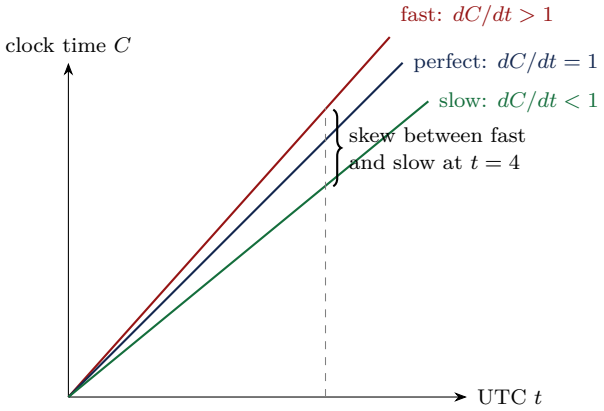


Sir's slide (w8-04). Simulating global time.

- **Clock skew**: the difference between the *values* of two clocks at the same instant (e.g. one reads 10:00:00.000, the other 10:00:00.030: skew 30 ms).
- **Clock drift**: the difference in their *rates*. A clock with **drift rate** ρ satisfies $1-\rho \le \frac{dC}{dt} \le 1+\rho$; quartz crystals have $\rho \approx 10^{-5}$ to 10^{-6} (10 to 1 ppm, about 1 s per 1–12 days).
- **Precision** π: keep any two clocks within π of *each other*: $|C_p(t) - C_q(t)| \le \pi$. Achieved

by **internal synchronisation**.

- **Accuracy** α: keep every clock within α of *real time (UTC)*: $|C_p(t) - t| \le \alpha$. Achieved by **external synchronisation**.



How often must clocks be resynchronised?

Two clocks each drift at most ρ from real time, in opposite directions in the worst case. After Δt seconds of real time they can differ by up to $2\rho\,\Delta t$. To keep them within δ of each other, resynchronise at least every

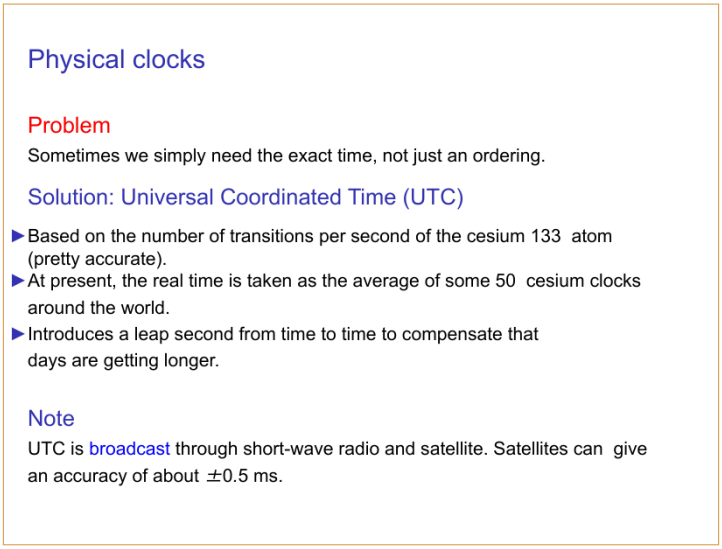
$$\Delta t_{\max} = \frac{\delta}{2\rho} \text{ seconds.}$$

Numerical

Questions. (D1) Quartz clocks with $\rho = 10^{-5}$ must stay within 1 ms of each other. How often must they be resynchronised? What is the worst skew after one hour without synchronisation? (D2) A clock gains 2 s per day. What is its drift rate in ppm? (D3) A clock is found 4 s ahead. Why not set it back, and how long does correcting it take if it is run 10% slow? (D4) With $\rho = 50$ ppm and resynchronisation every 10 minutes, what is the maximum skew?

Solutions. (D1) $\Delta t = \dfrac{10^{-3}}{2 \times 10^{-5}} = \mathbf{50\ s}$. After one hour: $2 \times 10^{-5} \times 3600 = \mathbf{0.072\ s} =$ 72 ms. (D2) $\rho = 2/86400 = 2.31 \times 10^{-5} = \mathbf{23.1\ ppm}$. (D3) Time must never run backwards: files and log entries would get timestamps *earlier* than ones already written (the `make` bug again). Instead slow the clock: if it advances 0.9 s per real second, it loses 0.1 s per second, so the 4 s are absorbed in $4/0.1 = \mathbf{40\ s}$ of real time. (D4) $2 \times 50 \times 10^{-6} \times 600 = 0.06$ s $= \mathbf{60\ ms}$.

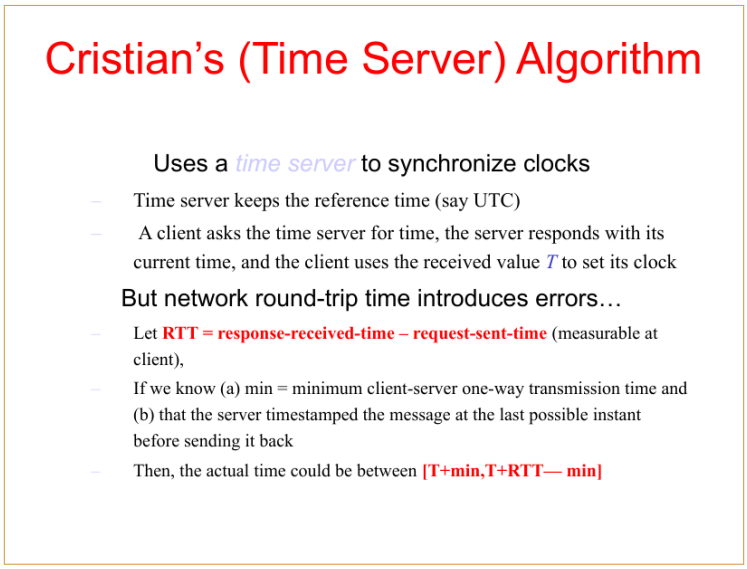
9.3 UTC



Sir's slide (w8-19). Physical clocks and UTC.

When we need the actual time, not just an order, we use **Coordinated Universal Time (UTC)**. The second is defined by transitions of the caesium-133 atom (9,192,631,770 per second); real time is the average of about 50 caesium clocks worldwide (TAI). Because the Earth's rotation is slowing, UTC inserts a **leap second** from time to time to stay in step with the Sun. UTC is broadcast by short-wave radio and satellite; GPS gives about ±0.5 ms (much better with good receivers).

9.4 Cristian's algorithm

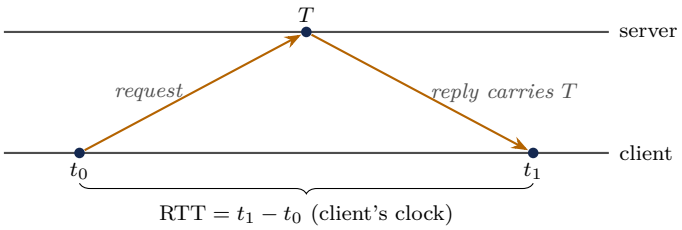


Sir's slide (w8-20). Cristian's (time server) algorithm.

A client asks a **time server** (which has UTC) for the time; the server replies with its time T. The reply is already old when it arrives, by the one-way delay. The client measures the

round-trip time $\mathrm{RTT} = t_{\text{receive}} - t_{\text{send}}$ on its own clock and assumes the reply took half of it:

$$C_{\text{new}} = T + \frac{\mathrm{RTT}}{2}.$$



Error bound. If the minimum one-way delay is min, and the server stamps T at the last moment before replying, the true time when the reply arrives lies in $[T + \min,\ T + \mathrm{RTT} - \min]$, an interval of width $\mathrm{RTT} - 2\min$. Choosing the midpoint $T + \mathrm{RTT}/2$ gives accuracy $\pm(\mathrm{RTT}/2 - \min)$. Short RTTs give accurate answers, so a client usually sends several requests and uses the one with the smallest RTT.

Consider a time server A. Another server B requests time from A at 10:37:640 (hr, min, 1/100 sec.), At 10:37:710, server B receives a reply from timeserver A with the timestamp of 10:37:375.

Find out the drift of B's clock with respect to the time-server A's clock (assume there is no processing time at the time-server for time service).

ANS:

RTT: Reply – Request = 610-540 = 70 1/100sec.

Adjusted local time: Server + RTT/2 = 375 + 35 = 410 1/100sec.

Drift: Adjusted local time – local time = 410 – 710 = -300 1/100sec. = -3 sec

Sir's slide (w8-21). Sir's worked Cristian example.

Numerical

Question (sir's slide). Server B requests the time from time server A at 10:37:640 (hr:min:1/100 s, i.e. B's seconds-and-hundredths field reads 6.40 s) and receives the reply at 10:37:710; the reply carries A's timestamp 10:37:375. No processing time at A. Find how far B's clock is off from A's.

Solution.

1. $\mathrm{RTT} = 710 - 640 = 70$ (hundredths of a second).
2. Correct time on arrival $= 375 + 70/2 = 375 + 35 = 410$.
3. B's clock reads 710 at that instant. Offset $= 410 - 710 = -300$ hundredths $= \mathbf{-3\ s}$.

B's clock is **3 seconds ahead** of A's; it must be *slowed* until it loses 3 s (not set back).

Watch out

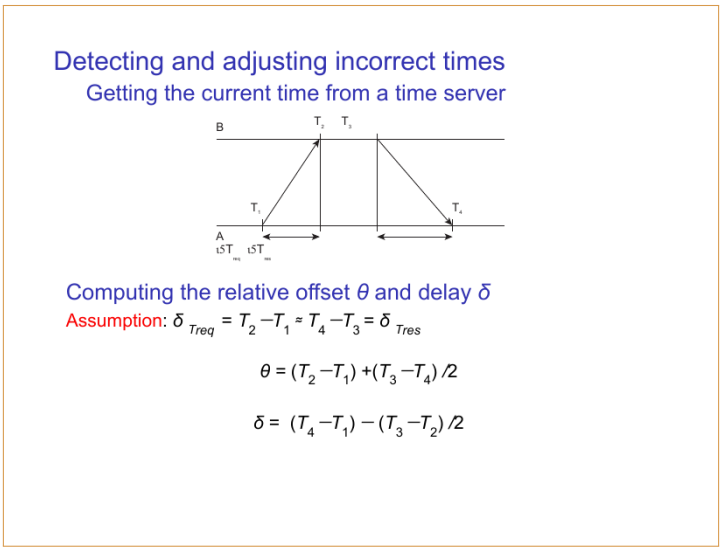
Two slips on that slide. The arithmetic line reads "$610 - 540 = 70$": the numbers are $710 - 640$. And the answer (-3 s) is the **offset** (skew) between the clocks, not the **drift**; drift is a *rate* (seconds per second). In the exam write "offset/skew $= -3$ s" and, if asked for "drift" as in the slide, mention that the slide uses the word loosely.

Numerical

Question. A client sends a request at 10:00:00.000 (its clock) and receives the reply at 10:00:00.020. The reply says 10:00:05.500. The minimum one-way delay is 3 ms. What should the client set, and how accurate is it?

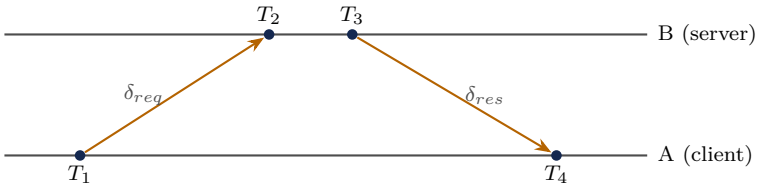
Solution. RTT $= 20$ ms. Set clock to $5.500 + 0.010 =$ **10:00:05.510**. True time is in $[5.500 + 0.003,\ 5.500 + 0.020 - 0.003] = [5.503,\ 5.517]$, so accuracy is $\pm(10 - 3) = \pm$**7 ms**.

9.5 The Network Time Protocol (NTP)



Sir's slide (w8-06). Detecting and adjusting incorrect times: offset and delay.

NTP improves on Cristian by using **four timestamps**, so the server's processing time drops out: T_1 client sends request (client clock), T_2 server receives it, T_3 server sends reply (server clock), T_4 client receives reply (client clock).



Offset θ and delay δ

Let θ be how far B's clock is ahead of A's. Write each one-way delay using the timestamps:

$$\delta_{req} = (T_2 - \theta) - T_1, \qquad \delta_{res} = T_4 - (T_3 - \theta).$$

Assume the two directions are equal, $\delta_{req} = \delta_{res}$:

$$T_2 - \theta - T_1 = T_4 - T_3 + \theta \implies \boxed{\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}}.$$

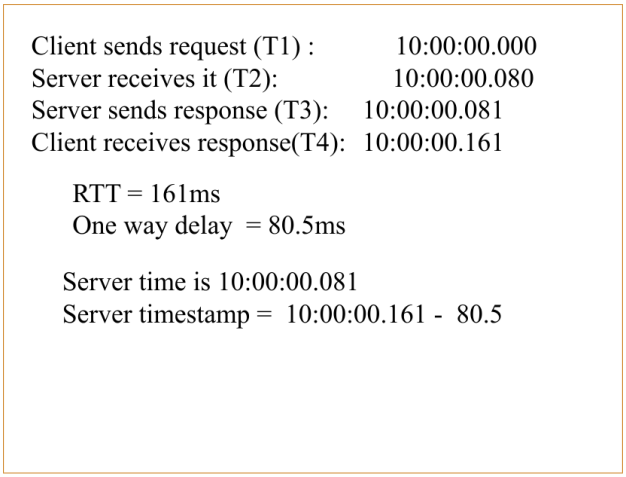
Adding the two delays gives the **round-trip delay** with the server's processing time removed:

$$\boxed{\delta = (T_4 - T_1) - (T_3 - T_2)}.$$

The true offset lies within $\theta \pm \delta/2$. NTP repeats the exchange, keeps the last 8 pairs, and uses the θ from the pair with the **smallest** δ (least room for asymmetry).

Watch out

Sir's slide writes $\delta = \big((T_4 - T_1) - (T_3 - T_2)\big)/2$. That halved value is the *one-way* delay (under the symmetry assumption); RFC 5905 defines the NTP delay as the full round trip without the /2. Say which one you mean.



Sir's slide (w8-05). Sir's NTP example.

Numerical

Question (sir's slide). $T_1 = 10{:}00{:}00.000$, $T_2 = 10{:}00{:}00.080$, $T_3 = 10{:}00{:}00.081$, $T_4 = 10{:}00{:}00.161$. Find the offset and delay.

Solution. In ms after 10:00:00: $T_1 = 0, T_2 = 80, T_3 = 81, T_4 = 161$. $\theta = \frac{(80-0)+(81-161)}{2} = \frac{80-80}{2} = \mathbf{0}$: the clocks agree. $\delta = (161-0)-(81-80) = \mathbf{160}$ ms round trip, one way 80 ms. (The slide's "RTT = 161, one way = 80.5" includes the 1 ms the server spent; NTP subtracts it.)

Numerical

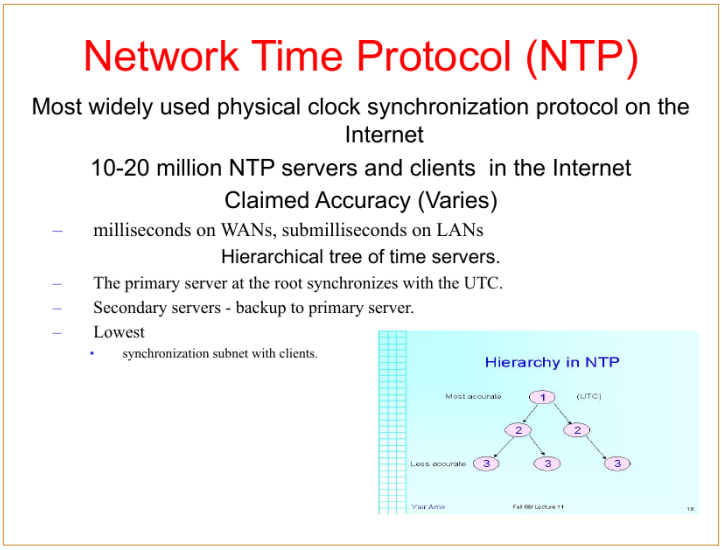
Question. $T_1 = 0$, $T_2 = 2050$, $T_3 = 2060$, $T_4 = 70$ (ms). Offset, delay, error bound?

Solution. $\theta = \frac{2050 + (2060 - 70)}{2} = \frac{2050 + 1990}{2} = \mathbf{2020}$ ms: the server is 2.02 s ahead, so the client adds 2.02 s (gradually). $\delta = 70 - 10 = \mathbf{60}$ ms. Error bound ±30 ms.

Numerical

Question (why symmetry matters). The clocks actually agree ($\theta_{\text{true}} = 0$). The request takes 150 ms, the reply 50 ms, server time negligible: $T_1 = 0$, $T_2 = 150$, $T_3 = 150$, $T_4 = 200$. What does NTP compute?

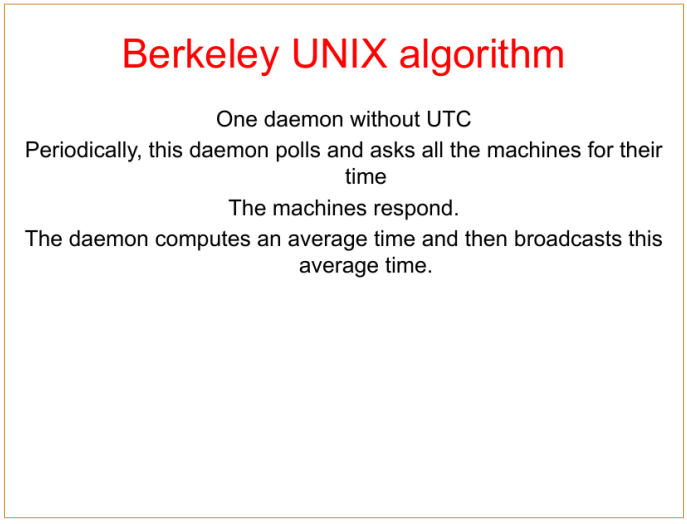
Solution. $\theta = \dfrac{150 + (150 - 200)}{2} = \mathbf{50}$ ms, although the true offset is 0. The error equals half the asymmetry, $(150 - 50)/2$, and stays within the bound $\pm\delta/2 = \pm 100$ ms. Asymmetric routes are NTP's main error source.



Sir's slide (w8-22). NTP hierarchy.

Hierarchy (strata). Stratum 0: reference clocks (atomic, GPS). Stratum 1 (primary servers): attached directly to them. Stratum 2: synchronise from stratum 1, and so on; the lowest levels are the synchronisation subnet of clients. Secondary servers back up primary ones. About 10–20 million NTP servers and clients; accuracy is milliseconds on WANs and sub-millisecond on LANs.

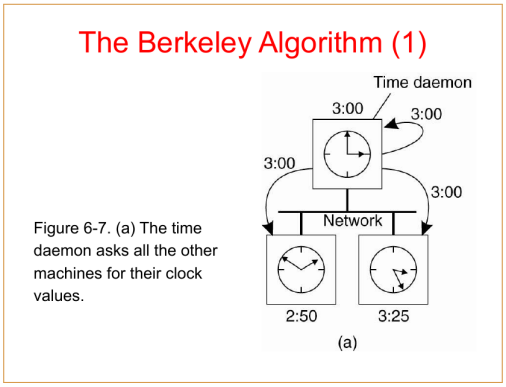
9.6 The Berkeley algorithm



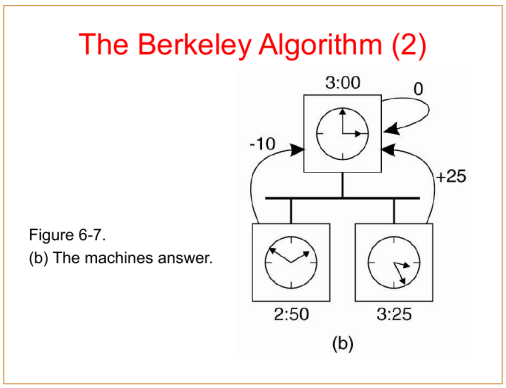
Sir's slide (w8-23). Berkeley UNIX algorithm.

Used when *no* machine has UTC and the goal is only to make clocks agree (internal synchronisation).

1. A **time daemon** (master) periodically polls every machine for its time.
2. Each machine answers (the daemon corrects each reading by half its RTT).
3. The daemon computes the **average** of all clocks, including its own.
4. It sends each machine the **adjustment** it needs (not the absolute time, which would be stale on arrival). Machines that are ahead slow down rather than jump back.



Sir's slide (w8-24). (a) The daemon asks for clock values.



Sir's slide (w8-25). (b) The machines answer with their offsets.

Numerical

Question (sir's figure). The daemon reads 3:00; the two machines read 2:50 and 3:25. Find the new time and each machine's adjustment.
Solution. Offsets from the daemon: 0, −10, +25 minutes. Average offset = $(0 - 10 + 25)/3 = +5$, so the agreed time is **3:05**. Adjustments = average − own offset: daemon +5, the 2:50 machine +15, the 3:25 machine **−20** (it slows down). Check: $3:00 + 5 = 2:50 + 15 = 3:25 - 20 = 3:05$.

Numerical

Question (fault-tolerant average). Daemon 10:00; others 10:02, 9:59, 10:01, and 10:30. Readings more than 10 minutes from the daemon are ignored as faulty. New time and adjustments?
Solution. Offsets $0, +2, -1, +1, +30$; drop +30. Average of the rest $= (0+2-1+1)/4 =$

+0.5 min, so the time becomes 10:00:30. Adjustments: daemon +0.5, −1.5, +1.5, −0.5, and the faulty machine $+0.5 - 30 = -29.5$ min (it is told to correct, but did not distort the average).

9.7 Comparing the three algorithms

	Cristian	**Berkeley**	**NTP**
Type	external (to UTC server)	internal (agreement)	external, hierarchical
Who has the right time	the server	nobody: the average	stratum 0/1 sources
Messages	request + reply	poll, replies, adjustments	4 timestamps, repeated
Formula	$T + \text{RTT}/2$	average of offsets	$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$
Weakness	single server; assumes symmetric delay	master is a single point of failure	asymmetric routes

Check yourself

Q. Skew vs drift?

A. Skew: difference in clock values. Drift: difference in clock rates.

Q. Precision vs accuracy?

A. Precision: clocks close to each other (internal sync). Accuracy: close to UTC (external sync).

Q. Why does Berkeley send adjustments, not the time?

A. The absolute time would be stale by the transmission delay; an adjustment is not.

Q. Why should a clock that is ahead not be set back?

A. Time must be monotonic; slow it down instead.

Q. Which NTP sample is trusted?

A. The one with the smallest round-trip delay.

10

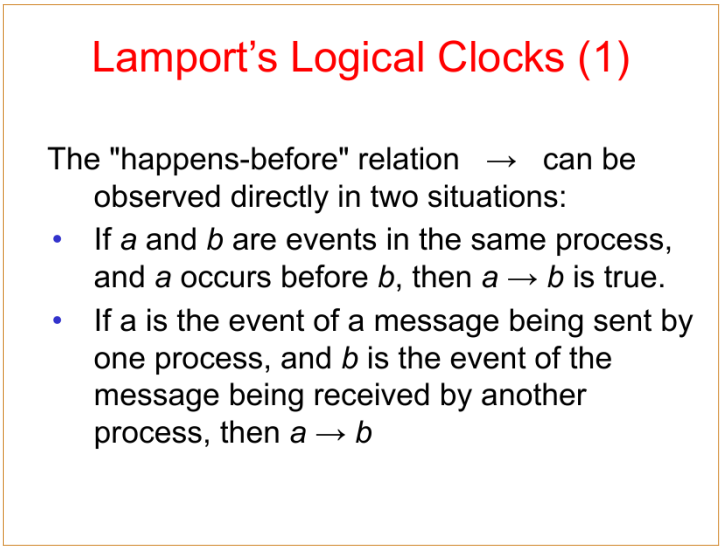
Logical Clocks: Lamport and Vector Clocks

Physical clocks can be made close but never equal. Often we do not need the exact time at all: we need every process to **agree on the order** in which events happened. Lamport (1978) showed how to do that with simple counters. Vector clocks (Fidge, Mattern, 1988) go further and detect when two events are *concurrent*. These produce the numericals you are most likely to get: "assign timestamps to the events in this diagram".

10.1 The happened-before relation



Sir's slide (w8-07). What matters is agreeing on the order of events.

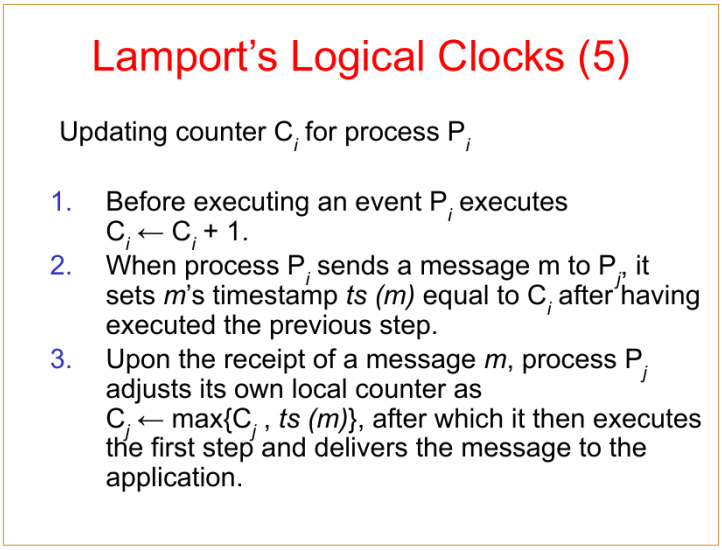


Sir's slide (w8-08). Happens-before.

> The **happened-before** relation $\rightarrow$ is the smallest relation such that:
>
> 1. if a and b are events in the **same process** and a occurs before b, then $a \rightarrow b$;
> 2. if a is the **sending** of a message and b is the **receipt** of that message, then $a \rightarrow b$;
> 3. it is **transitive**: $a \rightarrow b$ and $b \rightarrow c$ imply $a \rightarrow c$.
>
> Two events are **concurrent**, $a \parallel b$, if neither $a \rightarrow b$ nor $b \rightarrow a$.

$a \rightarrow b$ means "a could have influenced b": there is a path from a to b moving forward along process lines and along message arrows. If no such path exists either way, neither event could know about the other: they are concurrent, whatever the physical clocks say.

10.2 Lamport's logical clocks

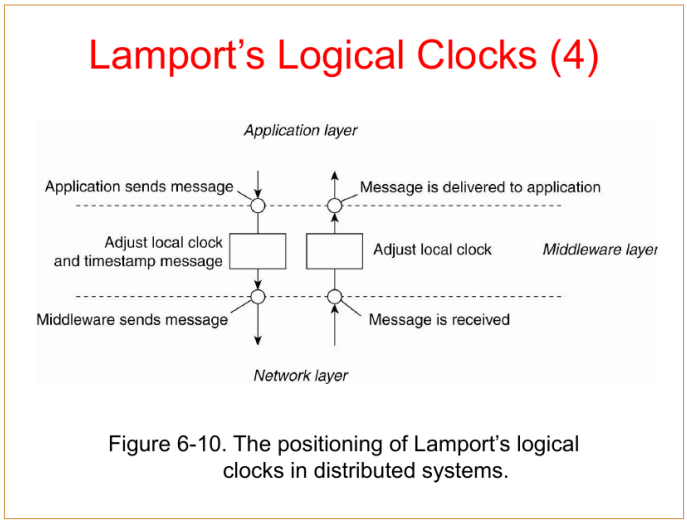


Sir's slide (w8-12). Updating the counter C_i.

Each process P_i keeps an integer counter C_i (initially 0).

1. **Before** executing any event (internal, send or receive), P_i does $C_i \leftarrow C_i + 1$.
2. When P_i **sends** message m, it attaches $ts(m) = C_i$ (the value after step 1).
3. When P_j **receives** m, it sets $C_j \leftarrow \max\{C_j,\ ts(m)\}$, then does step 1 and delivers the message.

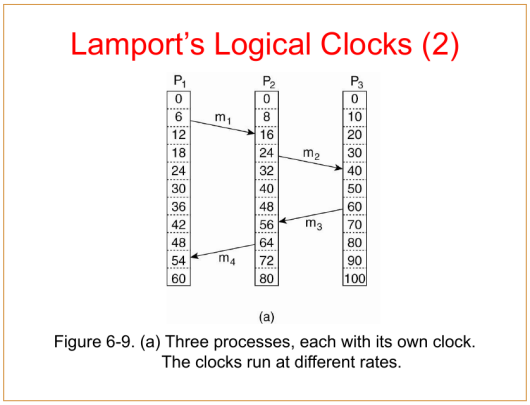
Clock condition: if $a \rightarrow b$ then $C(a) < C(b)$. The converse is **false**: $C(a) < C(b)$ does *not* imply $a \rightarrow b$.



Sir's slide (w8-11). Where Lamport clocks sit: in the middleware.

The adjustment happens in the **middleware layer**, between the application and the network: the middleware stamps outgoing messages and adjusts the local clock on receipt before the application sees the message.

The textbook figure, in numbers



Sir's slide (w8-09). (a) Three clocks running at different rates.

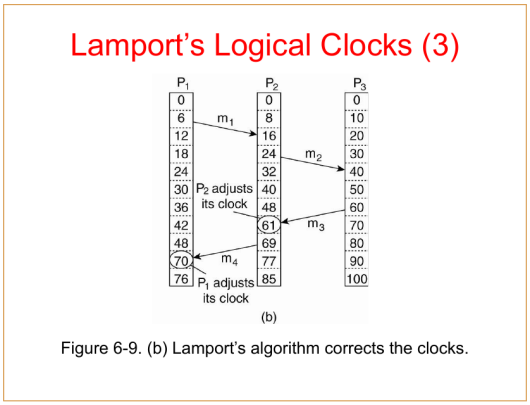
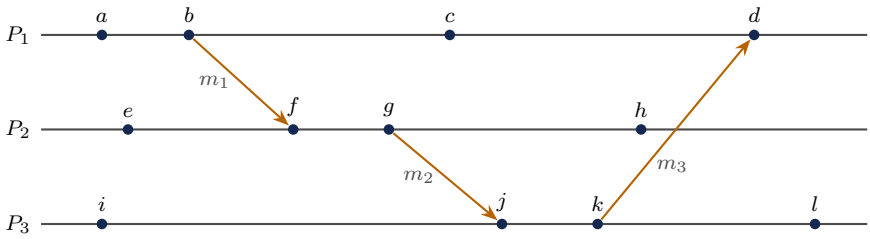


Figure 6-9. (b) Lamport's algorithm corrects the clocks.

Sir's slide (w8-10). (b) Lamport's algorithm corrects them.

P_1, P_2, P_3 tick by 6, 8 and 10 per step. m_1 (sent by P_1 at 6, received by P_2 at 16) and m_2 (P_2 at 24 $\rightarrow$ P_3 at 40) arrive at *later* readings: no problem. But m_3 leaves P_3 at 60 and arrives when P_2 reads 56, and m_4 leaves P_2 (after correction, at 69) and arrives when P_1 reads 54: a message would appear to arrive *before* it was sent. Lamport's rule fixes it: on receiving m_3, P_2 jumps to $\max(56, 60) + 1 = 61$, then continues 69, 77, 85; on receiving m_4, P_1 jumps to $\max(54, 69) + 1 = 70$, then 76. After correction every receive is stamped later than its send.

10.3 Worked example: the running diagram



Three processes, twelve events, three messages.

Numerical

Question. Assign Lamport timestamps and vector timestamps to all twelve events. Then decide: (i) c vs e; (ii) b vs k; (iii) h vs l; (iv) h vs d; (v) a vs l. Finally give a total order.
Lamport, process by process (each event: $+1$; each receive: $\max(\text{own}, ts) + 1$):

- P_1: $a = 1$, $b = 2$ (sends m_1 with 2), $c = 3$, $d = \max(3,\ ts(m_3) = 6) + 1 = \mathbf{7}$.
- P_2: $e = 1$, $f = \max(1, 2) + 1 = 3$, $g = 4$ (sends m_2 with 4), $h = 5$.
- P_3: $i = 1$, $j = \max(1, 4) + 1 = 5$, $k = 6$ (sends m_3 with 6), $l = 7$.

Vector $(V[P_1], V[P_2], V[P_3])$ (own entry $+1$; on receive, element-wise max with the message's vector first):

P_1: $a = (1,0,0)$	$b = (2,0,0)$	$c = (3,0,0)$	$d = (4,3,3)$
P_2: $e = (0,1,0)$	$f = (2,2,0)$	$g = (2,3,0)$	$h = (2,4,0)$
P_3: $i = (0,0,1)$	$j = (2,3,2)$	$k = (2,3,3)$	$l = (2,3,4)$

The three receives, written out: $f = \max\big((0,1,0),(2,0,0)\big) = (2,1,0)$, then own entry $+1$: $(2,2,0)$. $j = \max\big((0,0,1),(2,3,0)\big) = (2,3,1)$, then $+1$: $(2,3,2)$. $d = \max\big((3,0,0),(2,3,3)\big) = (3,3,3)$, then $+1$: $(4,3,3)$.
Relations (compare vectors: $u \to v$ iff $u \le v$ entry-wise and $u \neq v$; otherwise concurrent):

(i) $c = (3,0,0)$, $e = (0,1,0)$: neither $\le$ the other $\Rightarrow c \parallel e$. Yet Lamport gives $C(c) = 3 > C(e) = 1$: a bigger Lamport stamp does *not* mean "happened after".

(ii) $b = (2,0,0) \le k = (2,3,3) \Rightarrow b \to k$ (path $b \to f \to g \to j \to k$).

(iii) $h = (2,4,0)$, $l = (2,3,4)$: h has more of P_2, l more of $P_3 \Rightarrow h \parallel l$.

(iv) $h = (2,4,0)$, $d = (4,3,3)$: $4 > 3$ in P_2's entry, $3 > 0$ in P_3's $\Rightarrow h \parallel d$ (the message m_3 left P_3 before h existed).

(v) $a = (1,0,0) \le l = (2,3,4) \Rightarrow a \to l$.

Total order (sort by Lamport stamp, break ties by process number): $a(1,P_1)$, $e(1,P_2)$, $i(1,P_3)$, $b(2)$, $c(3,P_1)$, $f(3,P_2)$, $g(4)$, $h(5,P_2)$, $j(5,P_3)$, $k(6)$, $d(7,P_1)$, $l(7,P_3)$.

10.4 Partial order, total order, and totally ordered multicast

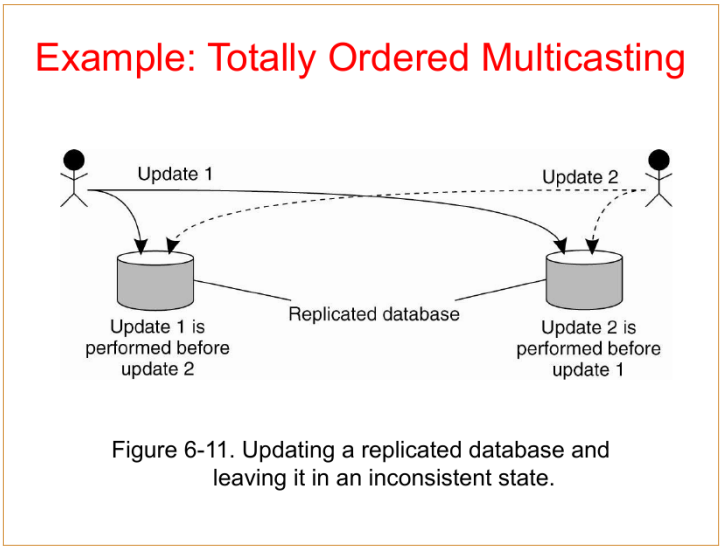
Partial order logical clocks assign timestamps to distributed events to create a causal, but not necessarily total, ordering, capturing the "happened-before" relationship without relying on physical clocks.

This establishes that one event could have caused another, but not all events can be compared (i.e., some events are concurrent).

Lamport's algorithm uses simple scalar timestamps for a partial ordering, while vector clocks provide a more complete ordering, enabling detection of concurrent events.

Sir's slide (w8-13). Lamport gives a partial (causal) order.

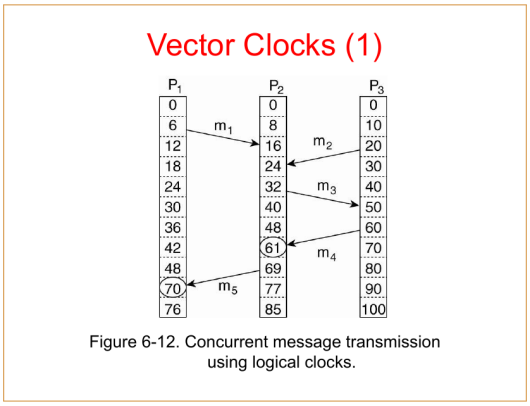
Lamport timestamps respect causality but cannot compare everything: concurrent events get stamps in some order that means nothing. To get a **total order** that every process agrees on, attach the process number and order by (timestamp, process id): $(C(a), i) < (C(b), j)$ iff $C(a) < C(b)$, or they are equal and $i < j$.



Sir's slide (w8-14). Figure 6-11: a replicated database left inconsistent.

Why total order matters. An account holds Rs. 1000 and is replicated in two cities. Update 1 adds Rs. 100; update 2 adds 1% interest. Replica A applies 1 then 2: $(1000 + 100) \times 1.01 = 1111$. Replica B applies 2 then 1: $1000 \times 1.01 + 100 = 1110$. The copies now disagree, and neither is wrong on its own. **Totally ordered multicast** fixes this: each update is multicast with a Lamport timestamp; each replica puts messages in a queue sorted by timestamp and acknowledges every message to everyone; a replica applies the message at the head of its queue only when it has been acknowledged by all processes (so no earlier-stamped message can still arrive). Every replica then applies updates in the same order.

10.5 Vector clocks



Sir's slide (w8-15). Figure 6-12: concurrency hidden by Lamport clocks.

Why Lamport is not enough. In Fig. 6-12 a process receives one message and later sends another: the receipt may well have caused the send, and the Lamport stamps are ordered. But a message that another process sent concurrently can also carry a smaller stamp than the later one, with no causal link at all. From the numbers alone you cannot tell "caused" from "merely earlier". Lamport stamps *lose* the causality information; vector clocks keep it.

Vector Clocks (2)

Vector clocks are constructed by letting each process P_i maintain a vector VC_i with the following two properties:

1. $VC_i[i]$ is the number of events that have occurred so far at P_i. In other words, $VC_i[i]$ is the local logical clock at process P_i.
2. If $VC_i[j] = k$ then P_i knows that k events have occurred at P_j. It is thus P_i's knowledge of the local time at P_j.

Sir's slide (w8-16). Vector clocks: the two properties.

Vector Clocks (3)

Steps carried out to accomplish property 2 of previous slide:

1. Before executing an event P_i executes $VC_i[i] \leftarrow VC_i[i] + 1$.
2. When process P_i sends a message m to P_j, it sets *m*'s (vector) timestamp *ts (m)* equal to VC_i after having executed the previous step.
3. Upon the receipt of a message m, process P_j adjusts its own vector by setting $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ for each *k*, after which it executes the first step and delivers the message to the application.

Sir's slide (w8-17). Vector clocks: the update rules.

Each process P_i keeps a vector VC_i of N counters with two properties: (1) $VC_i[i]$ is the number of events that have occurred so far at P_i (its own logical clock); (2) if $VC_i[j] = k$, then P_i knows that k events have occurred at P_j. **Rules:**

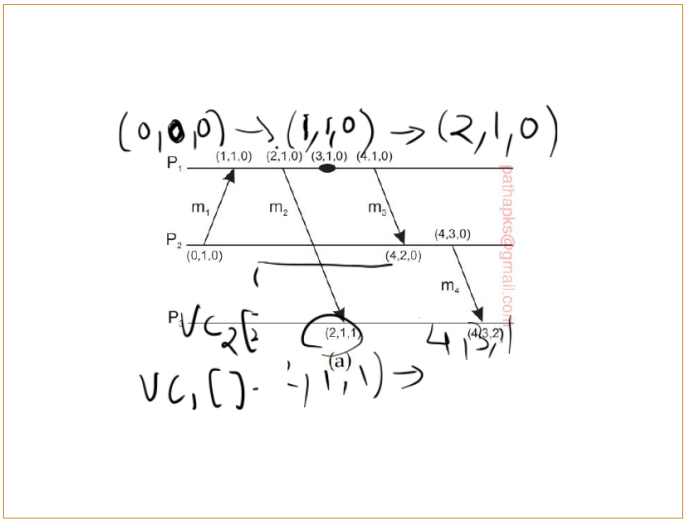
1. Before executing an event, P_i does $VC_i[i] \leftarrow VC_i[i] + 1$.
2. When sending m, attach $ts(m) = VC_i$ (after step 1).
3. On receiving m, P_j sets $VC_j[k] \leftarrow \max\{VC_j[k],\ ts(m)[k]\}$ for every k, then does step 1 and delivers.

Comparison: $u \le v$ iff $u[k] \le v[k]$ for all k; $u < v$ iff $u \le v$ and $u \ne v$. Then $a \to b \iff VC(a) < VC(b)$, and $a \parallel b$ iff neither $VC(a) \le VC(b)$ nor $VC(b) \le VC(a)$.

Mental model

A vector clock is "how many events I know about from each process". Event a happened before b exactly when b knows about everything a knew, and more. If each knows something the other does not, they are concurrent. Lamport squashes this vector into one

number and so loses the "each knows something the other does not" case.

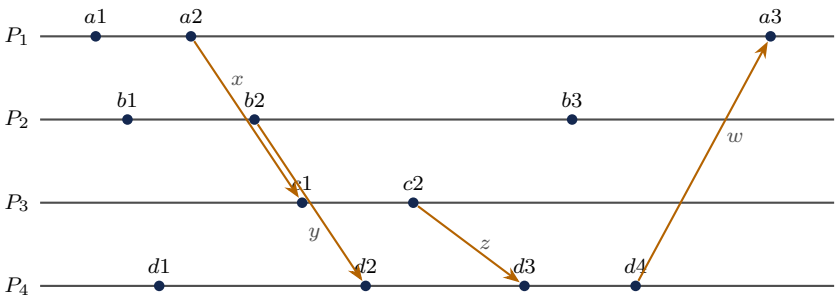


Sir's slide (w8-18). Sir's board example.

Numerical

Question (sir's board). P_2 sends m_1 to P_1; P_1 receives it, sends m_2 to P_3, has an internal event, sends m_3 to P_2; P_2 receives m_3 and sends m_4 to P_3; P_3 receives m_2 and then m_4. Give all vector timestamps. Did sending m_2 happen before sending m_4?

Solution. P_2 sends m_1: $(0,1,0)$. P_1 receives: $\max((0,0,0),(0,1,0)) = (0,1,0)$, then own $+1$: $(1,1,0)$. P_1 sends m_2: $(2,1,0)$; internal: $(3,1,0)$; sends m_3: $(4,1,0)$. P_2 receives m_3: $\max((0,1,0),(4,1,0)) = (4,1,0)$, own $+1$: $(4,2,0)$; sends m_4: $(4,3,0)$. P_3 receives m_2: $(2,1,1)$; receives m_4: $\max((2,1,1),(4,3,0)) = (4,3,1)$, own $+1$: $(4,3,2)$. $ts(m_2) = (2,1,0) < ts(m_4) = (4,3,0)$, so **sending m_2 happened before sending m_4**: P_3 should deliver m_2 before m_4 (causally ordered delivery), which it did.



A four-process exercise.

Numerical

Question. For the four-process diagram, compute Lamport and vector timestamps, then classify $a2$ vs $d3$, $b1$ vs $a1$, $b3$ vs $a3$, $b2$ vs $a3$, $c1$ vs $d2$.

Solution.

event	Lamport	vector	how
$a1, b1, d1$	1, 1, 1	$(1,0,0,0)$, $(0,1,0,0)$, $(0,0,0,1)$	first events
$a2$ (send x)	2	$(2,0,0,0)$	
$b2$ (send y)	2	$(0,2,0,0)$	
$c1$ (recv x)	$\max(0,2)+1=3$	$(2,0,1,0)$	
$d2$ (recv y)	$\max(1,2)+1=3$	$(0,2,0,2)$	
$b3$	3	$(0,3,0,0)$	
$c2$ (send z)	4	$(2,0,2,0)$	
$d3$ (recv z)	$\max(3,4)+1=5$	$(2,2,2,3)$	
$d4$ (send w)	6	$(2,2,2,4)$	
$a3$ (recv w)	$\max(2,6)+1=7$	$(3,2,2,4)$	

$a2 \to d3$ ($(2,0,0,0) \le (2,2,2,3)$). $b1 \parallel a1$. $b3 \parallel a3$: $b3 = (0,3,0,0)$ has a 3 for P_2 but $a3$ only knows 2 events of P_2, and $a3$ has entries $b3$ lacks. $b2 \to a3$ (path $b2 \to d2 \to d3 \to d4 \to a3$). $c1 \parallel d2$.

Drawing and solving a space–time diagram

1. One horizontal line per process, time running left to right. Dots for events, labelled.
2. Messages are slanted arrows from the send dot to the receive dot.
3. Fill timestamps **left to right in causal order**: you cannot stamp a receive until its send is stamped.
4. Lamport: $+1$ per event; at a receive take $\max(\text{own}, ts)+1$. Vector: own entry $+1$; at a receive take the element-wise max with the message's vector, then own $+1$.
5. To compare two events with vectors: write them one above the other and check every position. All $\le \Rightarrow \to$; mixed $\Rightarrow \parallel$.

	Lamport clock	**Vector clock**
Size per process	1 integer	N integers
$a \to b \Rightarrow$	$C(a) < C(b)$	$V(a) < V(b)$
Converse holds?	no	**yes**: $V(a) < V(b) \Rightarrow a \to b$
Detects concurrency?	no	yes
Total order?	yes, with process-id tie-break	no (only partial; can add tie-break)
Typical use	totally ordered multicast, mutual exclusion	causal multicast, conflict detection in replicated stores (e.g. Dynamo-style versioning)

Check yourself

Q. State the clock condition.
A. If $a \to b$ then $C(a) < C(b)$.

Q. Does $C(a) < C(b)$ imply $a \to b$?
A. No; they may be concurrent.

Q. Is $(2, 3, 1)$ before $(3, 3, 1)$?

A. Yes, every entry $\leq$ and not equal.

Q. $(2, 3, 1)$ vs $(3, 1, 1)$?

A. Concurrent.

Q. Why do replicas need totally ordered multicast?

A. Updates that do not commute (add vs multiply) must be applied in the same order everywhere.

Go deeper

Read. L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, CACM 21(7), 1978 (in `references/`).

Read. F. Mattern, *Virtual Time and Global States of Distributed Systems*, 1988 (in `references/`): vector clocks.

Read. Tanenbaum & van Steen, *Distributed Systems*, Ch. 6 : the figures on sir's slides.

Part IV

Exam kit

11

Insem 2 Exam Kit

Everything in one place for the last two days: the formulas, the slips on the slides, a practice set of numericals with answers, the conceptual question bank, the design questions, and a one-page summary.

11.1 Formula sheet

Cristian	$C = T + \text{RTT}/2$; true time $\in [T + \text{min},\ T + \text{RTT} - \text{min}]$; accuracy $\pm(\text{RTT}/2 - \text{min})$
NTP	$\theta = \frac{1}{2}[(T_2 - T_1) + (T_3 - T_4)]$; $\delta = (T_4 - T_1) - (T_3 - T_2)$; error $\leq \delta/2$
Berkeley	average the offsets (drop outliers); send each machine (average − own offset)
Drift	skew after $\Delta t \leq 2\rho\Delta t$; resync every $\delta/(2\rho)$; ppm $= \rho \times 10^6$
Lamport	$C \leftarrow C + 1$ per event; receive: $C \leftarrow \max(C, ts) + 1$; $a \to b \Rightarrow C(a) < C(b)$
Vector	$V[i]{+}{+}$ per event; receive: element-wise max then $V[i]{+}{+}$; $a \to b \iff V(a) < V(b)$
Popek–Goldberg	sensitive $\subseteq$ privileged $\Rightarrow$ trap-and-emulate VMM exists
Overhead	time $= N \cdot c_{\text{native}} + N_{\text{sensitive}} \cdot c_{\text{trap}}$
Live migration	round k copies D_{k-1}, takes D_{k-1}/B, dirties $D \cdot D_{k-1}/B$; downtime = last/B
Availability	series $\prod A_i$; parallel $1 - (1 - a)^n$; downtime $= (1 - A) \times 8760$ h
Capacity	VMs $= \lceil \text{load}/(\text{per-VM capacity} \times \text{target util.}) \rceil$
Queues	busy workers $= \lambda S$; stable iff workers $> \lambda S$; Little $L = \lambda W$; backlog $= (\lambda - \mu)t$
CIDR	$/n$ has 2^{32-n} addresses; AWS usable $= 2^{32-n} - 5$; splitting $/a$ into $/b$ gives 2^{b-a} subnets

11.2 Slips on the slides, all in one place

1. **Week 8, slide 21 (Cristian)**: "610 − 540 = 70" should be 710 − 640 = 70; the −3 s result is an *offset/skew*, not a drift (drift is a rate).

2. **Week 8, slide 6 (NTP)**: $\delta = ((T_4 - T_1) - (T_3 - T_2))/2$ is the one-way delay; NTP's delay (RFC 5905) is the round trip, without /2.

3. **Week 8, slide 5**: "RTT = 161 ms" includes the server's 1 ms; NTP's delay is 160 ms; offset 0.

4. **Week 5 Lec 1, slide 32**: \$32.2B is Microsoft's cloud revenue and \$25.7B is AWS's (2018); the slide places \$25.7B under GCP.

5. **Friend's notes**: "sensitive instructions are those that can trap" mixes the two classes: *privileged* instructions trap; *sensitive* ones touch or depend on the machine's configuration.

11.3 Practice numericals (answers verified in `scripts/numericals.py`)

1. **Cristian.** Request sent at 8:00:00.100, reply received at 8:00:00.140 carrying 8:00:03.000. Set time and offset? *Answer*: set 8:00:03.020; offset = $3.020 - 0.140 = +2.880$ s (client is behind).

2. **NTP.** $T_1 = 100$, $T_2 = 250$, $T_3 = 260$, $T_4 = 170$ ms. *Answer*: $\theta = \frac{150+90}{2} = 120$ ms; $\delta = 70 - 10 = 60$ ms.

3. **Berkeley.** Daemon 9:00; others 9:05, 8:52, 9:07. *Answer*: offsets $0, +5, -8, +7$, average $+1$, new time 9:01; adjustments $+1, -4, +9, -6$.

4. **Drift.** $\rho = 2 \times 10^{-6}$, keep within 10 ms. *Answer*: resync every $10^{-2}/(4 \times 10^{-6}) = 2500$ s.

5. **Vectors.** Relate $(1,2,0)$ & $(1,3,1)$; $(2,1,0)$ & $(1,2,0)$; $(0,0,3)$ & $(1,0,3)$. *Answer*: $\rightarrow$; $\|$; $\rightarrow$.

6. **Availability.** Two web servers at 95% in parallel, then a DB at 99%. *Answer*: $(1-0.05^2) \times 0.99 = 0.9975 \times 0.99 = 0.9875$.

7. **CIDR.** How many addresses in `192.168.10.0/26`, how many usable on AWS, how many /26 in a /24? *Answer*: 64; 59; 4.

8. **Live migration.** 2048 MB, 100 MB/s, dirty 40 MB/s, stop below 50 MB. *Answer*: rounds copy 2048, 819.2, 327.7, 131.1, 52.4 MB (5 rounds); 21.0 MB left: downtime ≈ 210 ms; total ≈ 34 s.

9. **Queue.** $\lambda = 5$ jobs/s, 2 s each, 12 workers. *Answer*: 10 busy on average, utilisation 83.3%, stable.

10. **Overhead.** 10^8 instructions, 1% sensitive, 500 cycles per trap, 1 cycle otherwise. *Answer*: $10^8 + 10^6 \times 500 = 6 \times 10^8$: 6× slower. Too many sensitive instructions destroy efficiency.

11. **Popek–Goldberg.** An instruction reads the current privilege level without trapping. Is the architecture classically virtualisable? *Answer*: no: it is behaviour-sensitive (mode-sensitive) but not privileged, so sensitive $\not\subseteq$ privileged.

12. **Lamport total order.** Events x on P_2 with $C = 4$ and y on P_1 with $C = 4$. Which comes first in the total order? *Answer*: y (tie broken by the smaller process id).

11.4 Conceptual question bank

Q. Define elastic computing and give an example.
A. Leasing exactly the capacity needed and changing it at any time; e.g. a seasonal company adds servers for five peak months.

Q. Compare IaaS, PaaS and SaaS with one example each.
A. IaaS: you manage OS and up (EC2). PaaS: you manage code and data (App Service). SaaS: you manage only your account (Office 365).

Q. Private vs public vs hybrid vs multi-cloud.
A. Chapter 1: owner, motivation, trade-off for each.

Q. What is a hypervisor? Type 1 vs type 2.
A. Software that creates and manages VMs; type 1 on hardware (ESXi, Xen, Hyper-V, KVM), type 2 on a host OS (VirtualBox, VMware Workstation).

Q. Explain the three approaches to virtualisation.
A. Emulation (interprets, slow, any ISA), para-virtualisation (modified guest, hypercalls, fast), full virtualisation (unmodified guest, fast).

Q. List the types of virtualisation.
A. By technique: full (BT), para, hardware-assisted, OS-level, emulation. By resource: server, memory, storage, network, desktop, application, I/O.

Q. Why does a VM run at near hardware speed?
A. Code runs directly on the CPU; only operations beyond the current privilege trap to a higher level (three privilege levels).

Q. What is a virtual I/O device? Two ways to build one.
A. Software that answers the guest's bus operations; invent a clean imaginary device (virtio) or emulate a real one (e1000).

Q. Explain live migration.
A. Pre-copy (copy memory while running, repeat dirty pages), stop-and-copy (pause, copy the rest), post-copy (send registers/state, resume).

Q. State and explain the Popek–Goldberg theorem.
A. Chapter 3: properties, classes, theorem, proof idea.

Q. Explain trap-and-emulate with a diagram.
A. Guest runs deprivileged; privileged instruction traps; hypervisor emulates on virtual state; guest resumes.

Q. Why was x86 hard to virtualise and how was it fixed?
A. Sensitive non-privileged instructions (POPF, SGDT, …); binary translation, para-virtualisation, VT-x/AMD-V.

Q. List the requirements for virtualisation.
A. Privilege modes, sensitive $\subseteq$ privileged (or a fix), memory protection, trap mechanism, I/O virtualisation, the three properties.

Q. What is a VPC? Name its components.
A. Isolated virtual network; CIDR, subnets, route tables, IGW, NAT gateway, security groups,

NACLs, peering/transit gateway.

Q. Security group vs network ACL.

A. Instance vs subnet; stateful vs stateless; allow-only vs allow+deny.

Q. Explain the layered cloud architecture.

A. Infrastructure (IaaS), platform (PaaS), application (SaaS); higher layers need more provider work.

Q. What is a market-oriented cloud architecture?

A. Resources allocated by economic models and SLAs; SLA resource allocator with admission control, pricing, accounting, VM monitor, dispatcher.

Q. List six cloud design challenges.

A. Chapter 5.

Q. Name and compare five architecture styles.

A. N-tier, Web-Queue-Worker, microservices, event-driven, big data/compute: Chapter 6.

Q. Layer vs tier with an example.

A. Phone contacts app: 3 layers, 1 tier; web app: 3 layers, 3 tiers.

Q. Closed vs open layers.

A. Closed: call only the next layer down. Open: any lower layer.

Q. Role of Traffic Manager.

A. DNS-level routing across regions (priority, weighted, performance, geographic), health monitoring, automatic failover; active–passive hot/cold, active–active.

Q. Steps to develop an N-tier architecture.

A. Purpose; business processes; requirements; software architecture; number of tiers.

Q. Explain Web-Queue-Worker and when to use it.

A. Web front end + worker via async queue; PaaS; simple domains with long-running or batch work.

Q. Skew vs drift; precision vs accuracy; internal vs external sync.

A. Chapter 9.

Q. Explain Cristian's, Berkeley and NTP; compare.

A. Chapter 9, comparison table.

Q. Define happened-before and concurrency.

A. Same-process order, send-before-receive, transitivity; concurrent if neither.

Q. Lamport vs vector clocks.

A. Scalar vs vector; Lamport cannot detect concurrency; vector can.

Q. What is totally ordered multicast and why is it needed?

A. Every replica delivers messages in the same (timestamp, id) order, so non-commuting updates leave replicas identical.

11.5 Design questions to practise drawing

1. Online bookstore (sir's five steps) — Chapter 7.
2. University result portal with a two-day spike — read replicas, scheduled scaling, cache.
3. Food delivery app — N-tier + queue for notifications and rider assignment.
4. Migrating a bank's on-premises app — N-tier on VMs, perimeter network, VPN, multi-region priority routing.
5. Video-sharing site — Web-Queue-Worker for transcoding, CDN for playback, blob storage.
6. Photo upload with thumbnails — Web-Queue-Worker, queue-length autoscaling.
7. A three-tier VPC across two AZs with CIDR plan — Chapter 4.

For each: logical diagram, physical diagram (VPC, AZs, subnets, LB, ASG, DB primary/standby), five lines of justification (scaling, availability, security, caching, cost).

11.6 Last-night summary

One page

Virtualisation. VM = software computer managed by a hypervisor (type 1 bare metal, type 2 hosted). Emulation slow; para needs a modified guest; full runs unmodified guests. Three privilege levels; exceeding yours traps upward. Virtual devices; VM as a digital object; three-phase live migration. **Popek–Goldberg**: equivalence, resource control, efficiency; sensitive $\subseteq$ privileged $\Rightarrow$ trap-and-emulate works. x86 broke it (POPF); fixed by BT, para, VT-x. **VPC**: CIDR, public/private subnets, route tables (longest prefix), IGW, NAT, SG (stateful) vs NACL (stateless).
Architecture. Goals: scalability, virtualisation, efficiency, reliability. Layers: IaaS, PaaS, SaaS. Styles: N-tier (IaaS, layers vs tiers, migrations), Web-Queue-Worker (PaaS, async queue, long jobs), microservices, event-driven, big data. Draw: VPC, 2 AZs, 3 subnet rows, LB, ASGs, DB primary/standby, SG chain.
Time. Skew (values) vs drift (rates); resync every $\delta/2\rho$. Cristian $T + \mathrm{RTT}/2$; NTP θ, δ from four stamps; Berkeley averages. Lamport max $+1$, total order by (C, id); vectors detect concurrency; totally ordered multicast keeps replicas consistent.

11.7 Sources

- D. Comer, *The Cloud Computing Book*, Chapters 2, 3 and 5 (course textbook; the shared PDFs).
- K. Hwang, G. Fox, J. Dongarra, *Distributed and Cloud Computing*, Chapter 4 (Week 6 figures).
- G. Popek, R. Goldberg, *Formal Requirements for Virtualizable Third Generation Architectures*, CACM 1974.

- J. Robin, C. Irvine, *Analysis of the Intel Pentium's Ability to Support a Secure VMM*, 2000.
- P. Barham et al., *Xen and the Art of Virtualization*, SOSP 2003 (in `references/`).
- L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, 1978 (in `references/`).
- F. Mattern, *Virtual Time and Global States of Distributed Systems*, 1988 (in `references/`).
- F. Cristian, *Probabilistic Clock Synchronization*, 1989 (in `references/`); RFC 5905 (NTPv4).
- A. Tanenbaum, M. van Steen, *Distributed Systems*, Chapter 6 (Week 8 figures).
- Microsoft Azure Architecture Center: *N-tier architecture style*; *Web-Queue-Worker architecture style*.
- NIST SP 800-145, *The NIST Definition of Cloud Computing* (in `references/`).